

Ingineria Programării

Cursul 4 – 18 Martie 2025

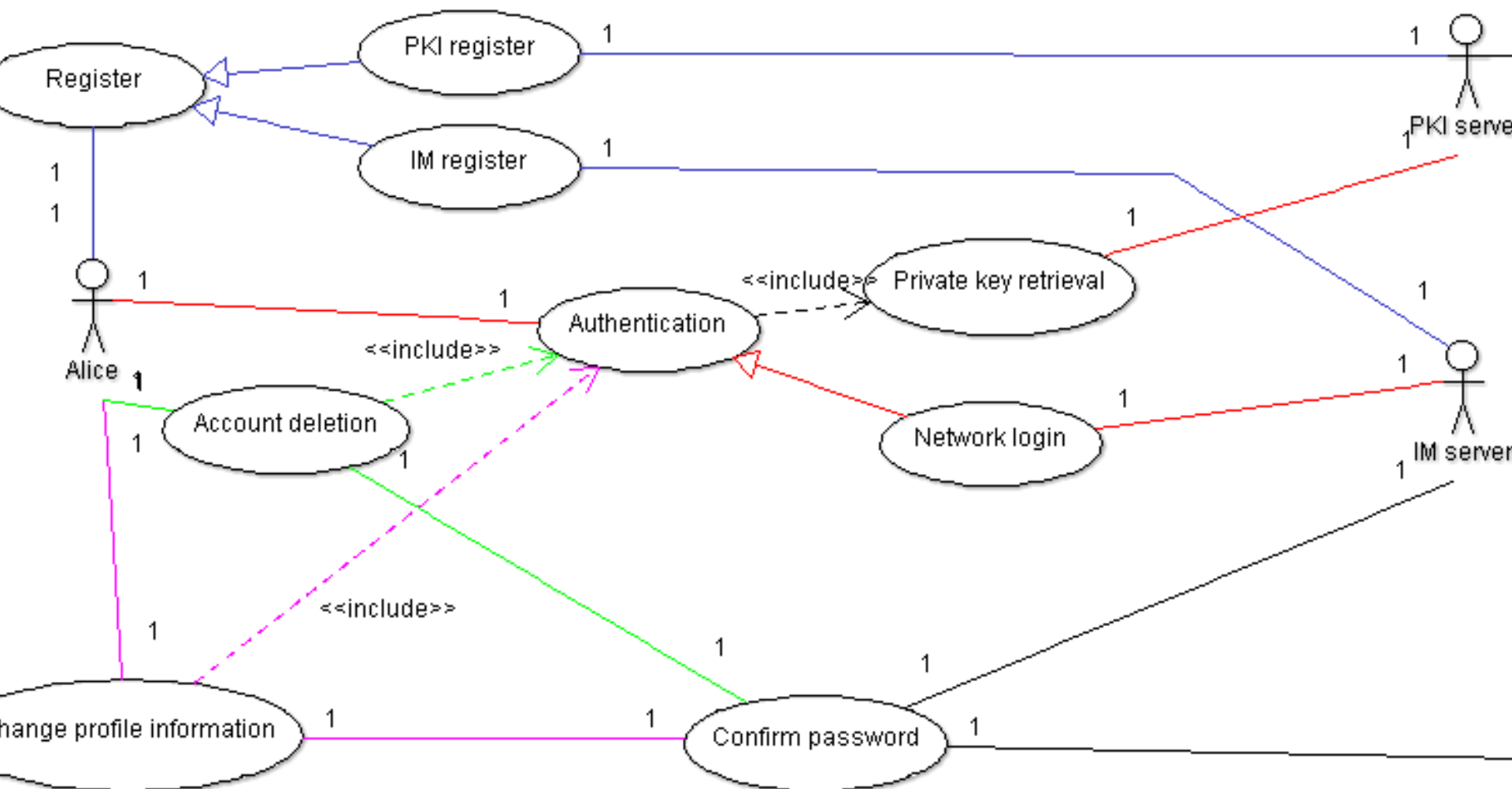
adiftene@info.uaic.ro

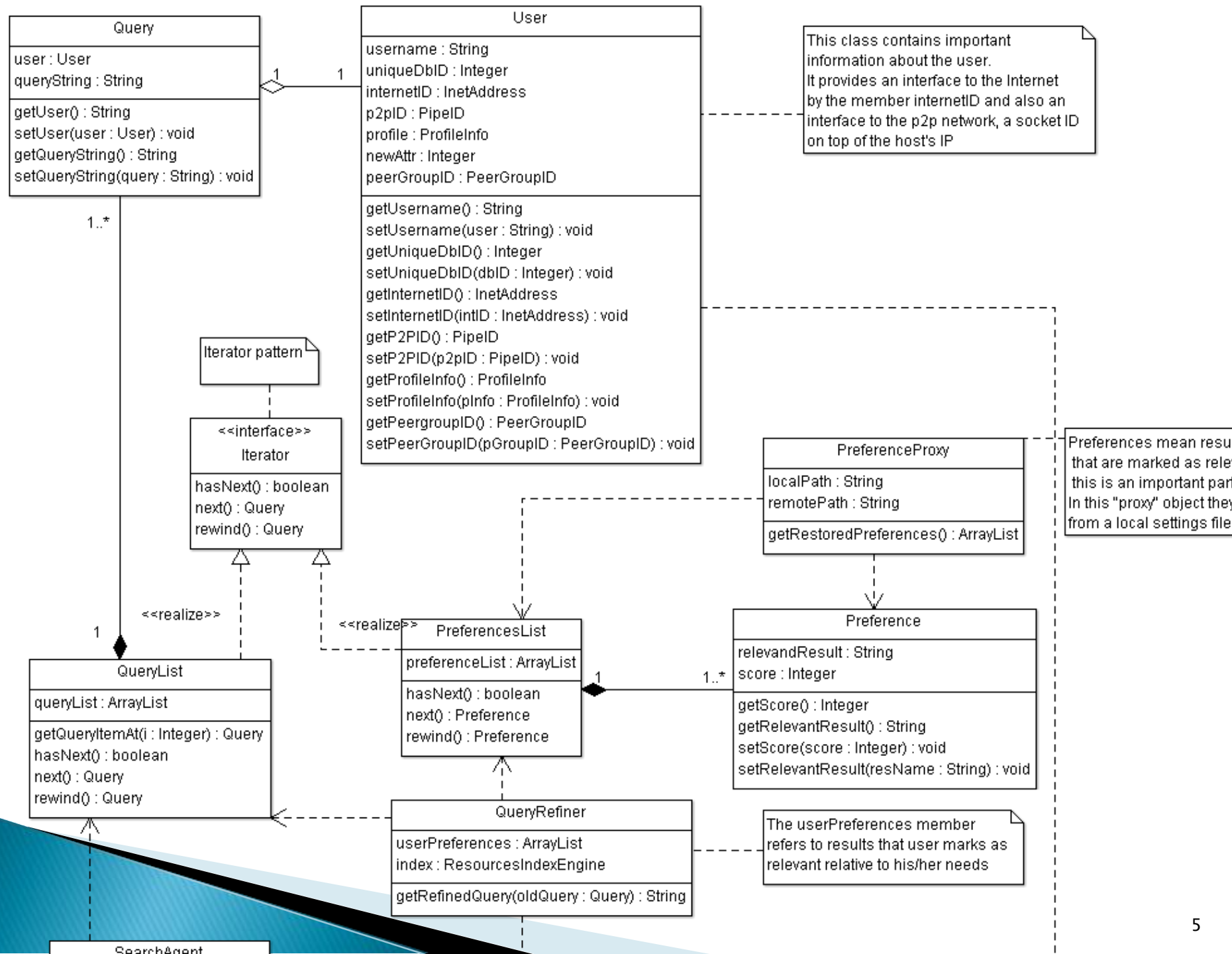
Cuprins

- ▶ Din Cursurile trecute...
- ▶ Diagrame UML – Exemple
- ▶ C4 Model
- ▶ Forward & Reverse Engineering

Din cursurile trecute...

- ▶ Diagrame
- ▶ Diagrame UML
- ▶ Diagrame Use Case
- ▶ Diagrame de Clase





UML2.0 – 13 Tipuri de Diagrame

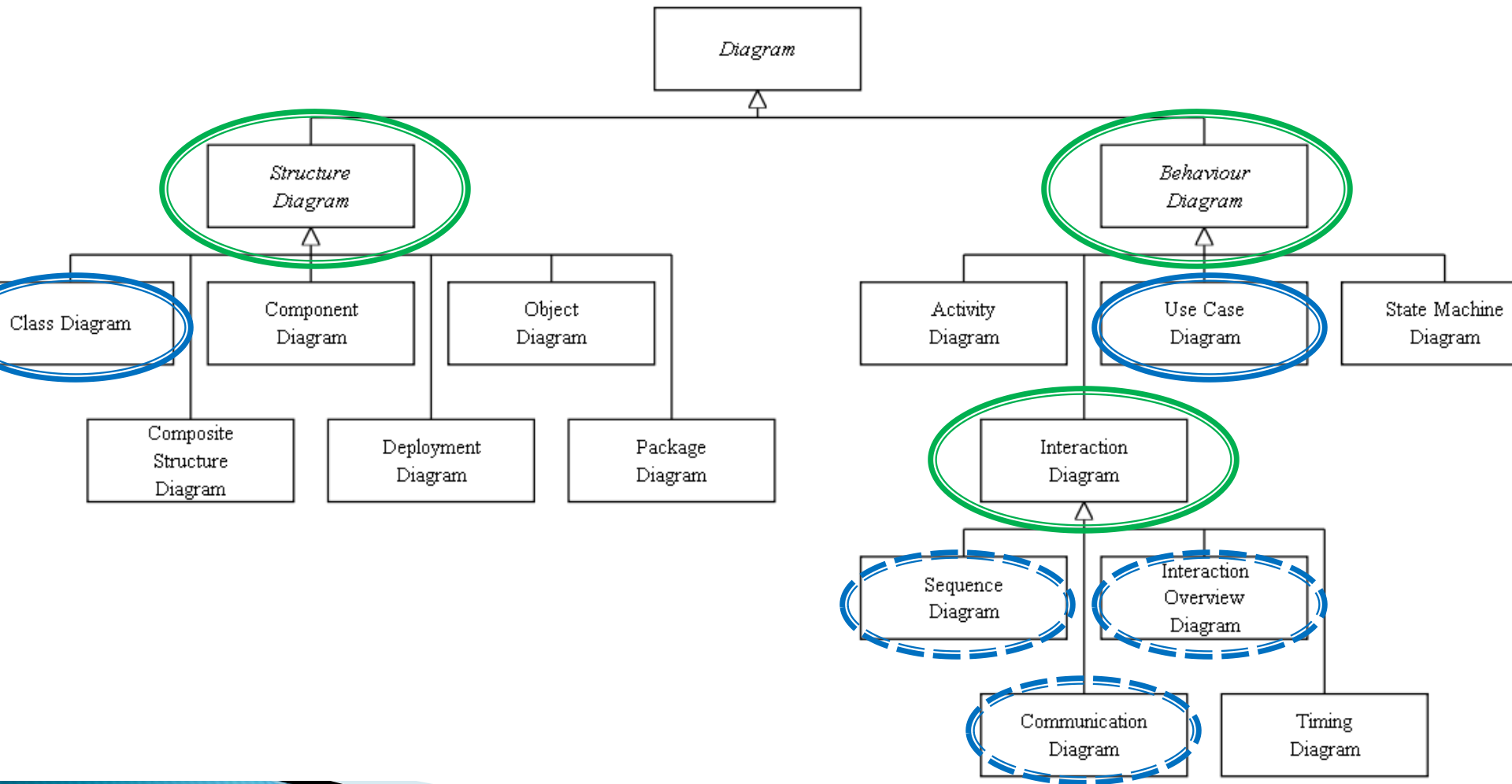
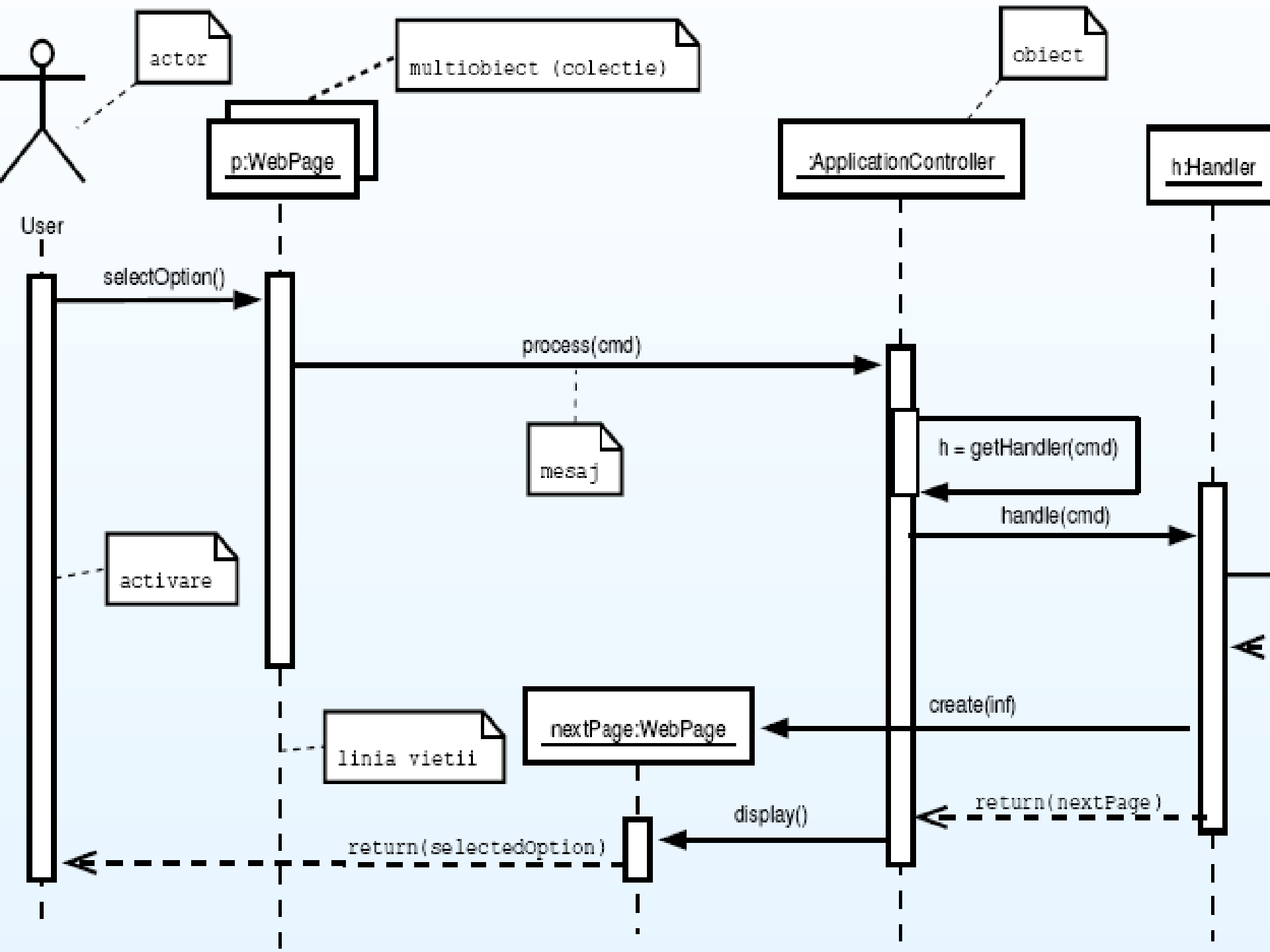
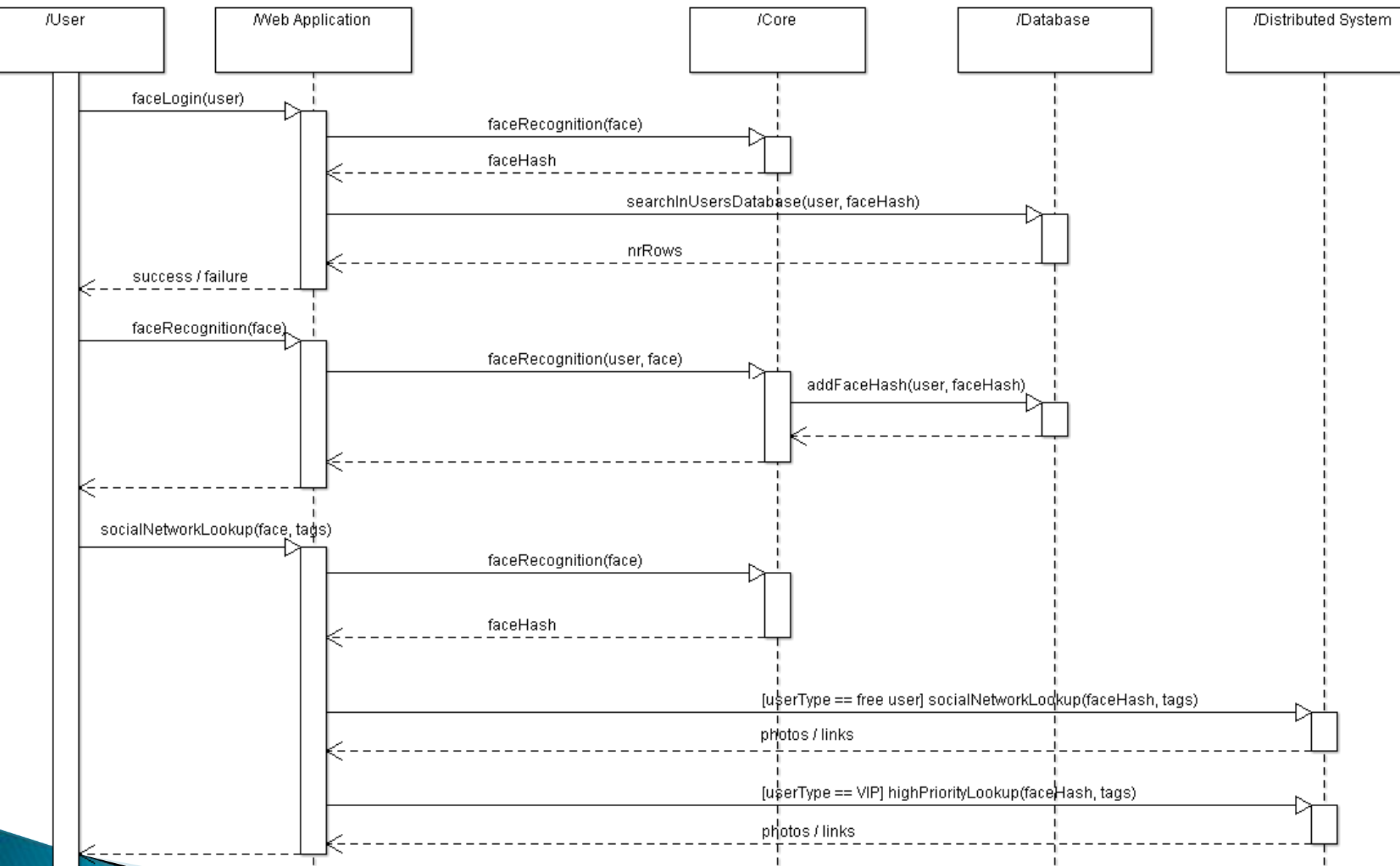


Diagrama de Secvență

- ▶ **Diagrama de secvență** curprinde secvența acțiunilor care au loc în sistem, invocarea metodelor fiecărui obiect ca și ordinea în timp în care aceste invocări au loc
- ▶ O diagramă de secvență este bidimensională
 - Pe axa verticală se prezintă viața obiectului
 - linia vieții obiectelor (grafic: linie punctată)
 - perioada de activare în care un obiect preia controlul execuției (grafic: dreptunghi pe linia vieții)
 - Pe axa orizontală se arată secvența creării sau invocărilor
 - mesaje ordonate în timp (grafic: săgeți)

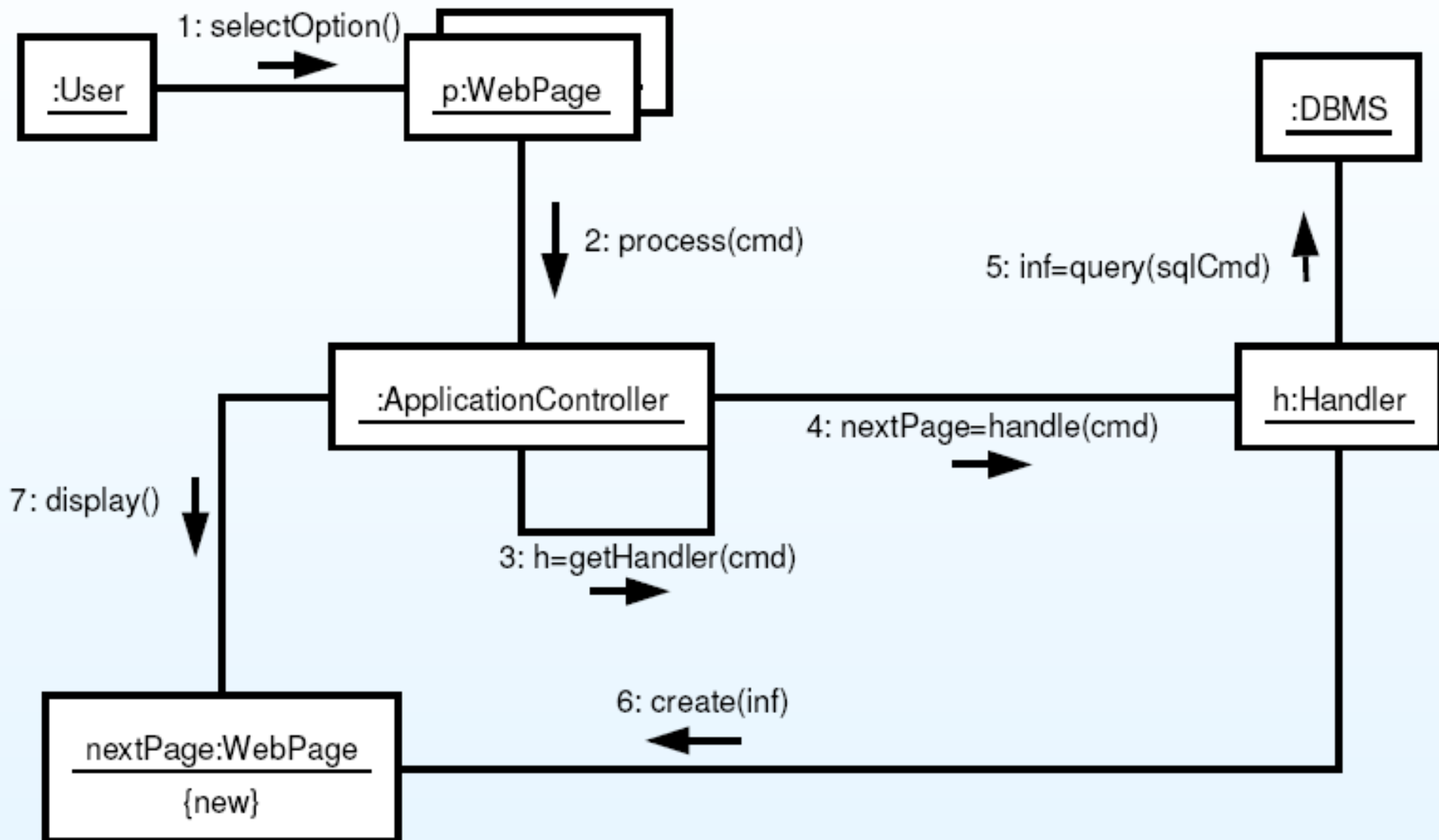




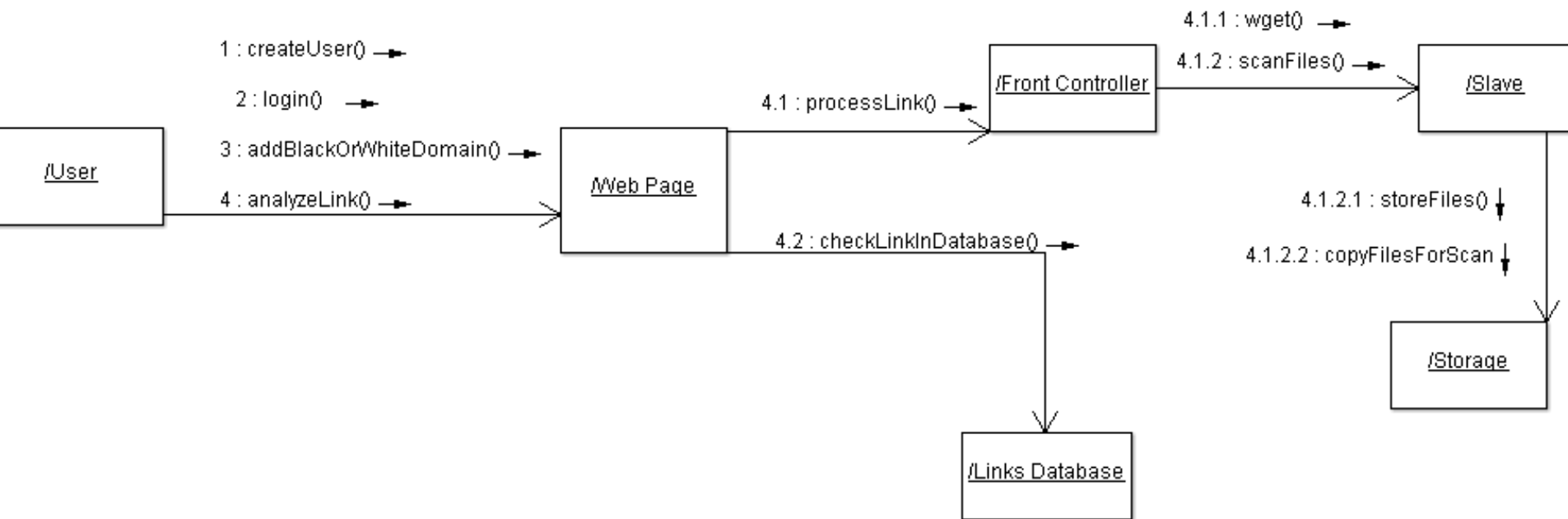
Diagramă de Colaborare

- ▶ Pune accentul pe organizarea structurală a obiectelor care participă la interacțiune
- ▶ Ilustrează mai bine ramificări complexe, iterații și comportament concurent
- ▶ Poate conține:
 - Obiecte, clase, actori
 - Legături între acestea
 - Mesaje

Exemplul 1



Exemplul 3

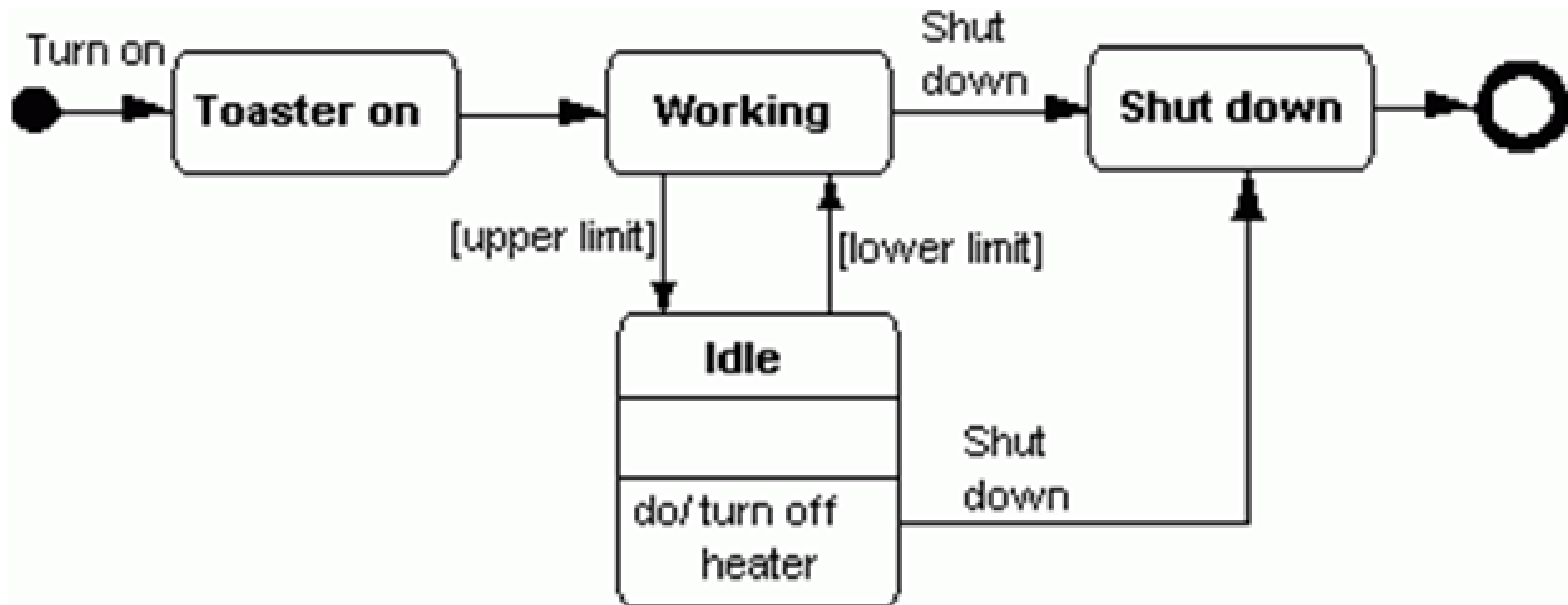
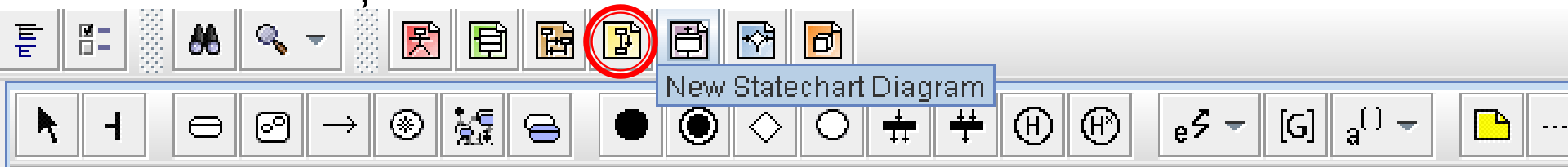


Diagrame comportamentale

- ▶ Diagrame de stări, diagrame de activități
- ▶ Elemente de bază
 - Eveniment
 - Acțiune
 - Activitate

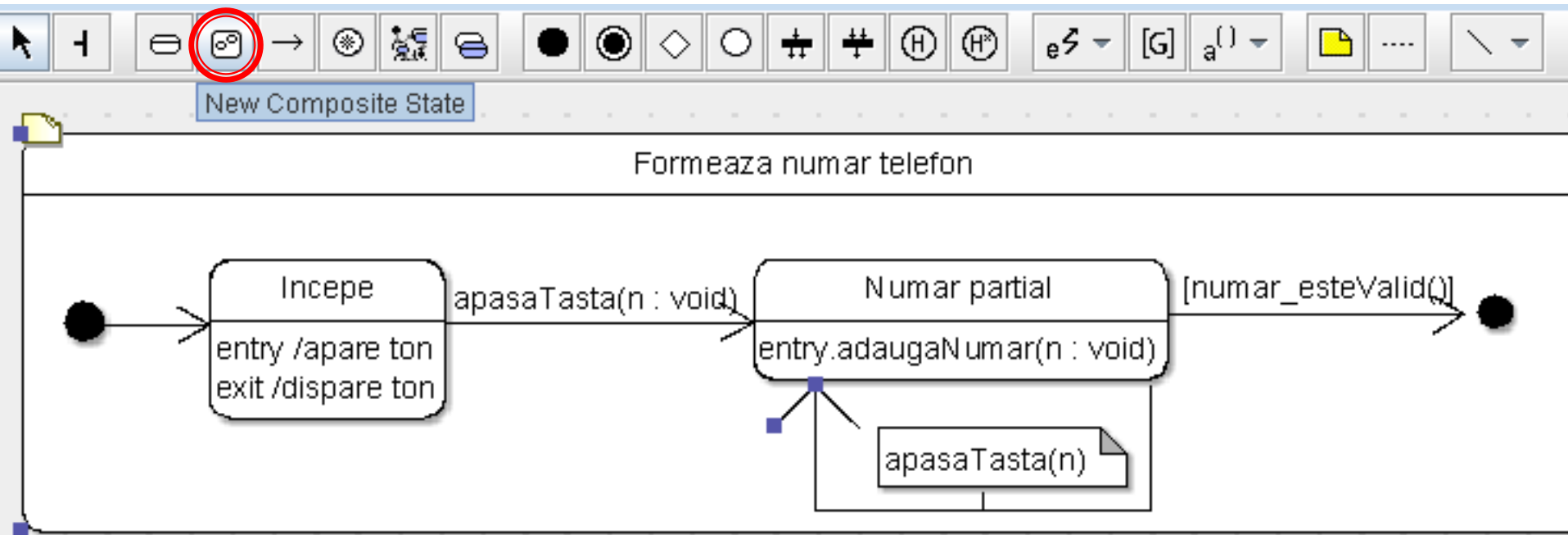
Diagramă de Stări

- ▶ Conține:
 - Stări
 - Tranziții



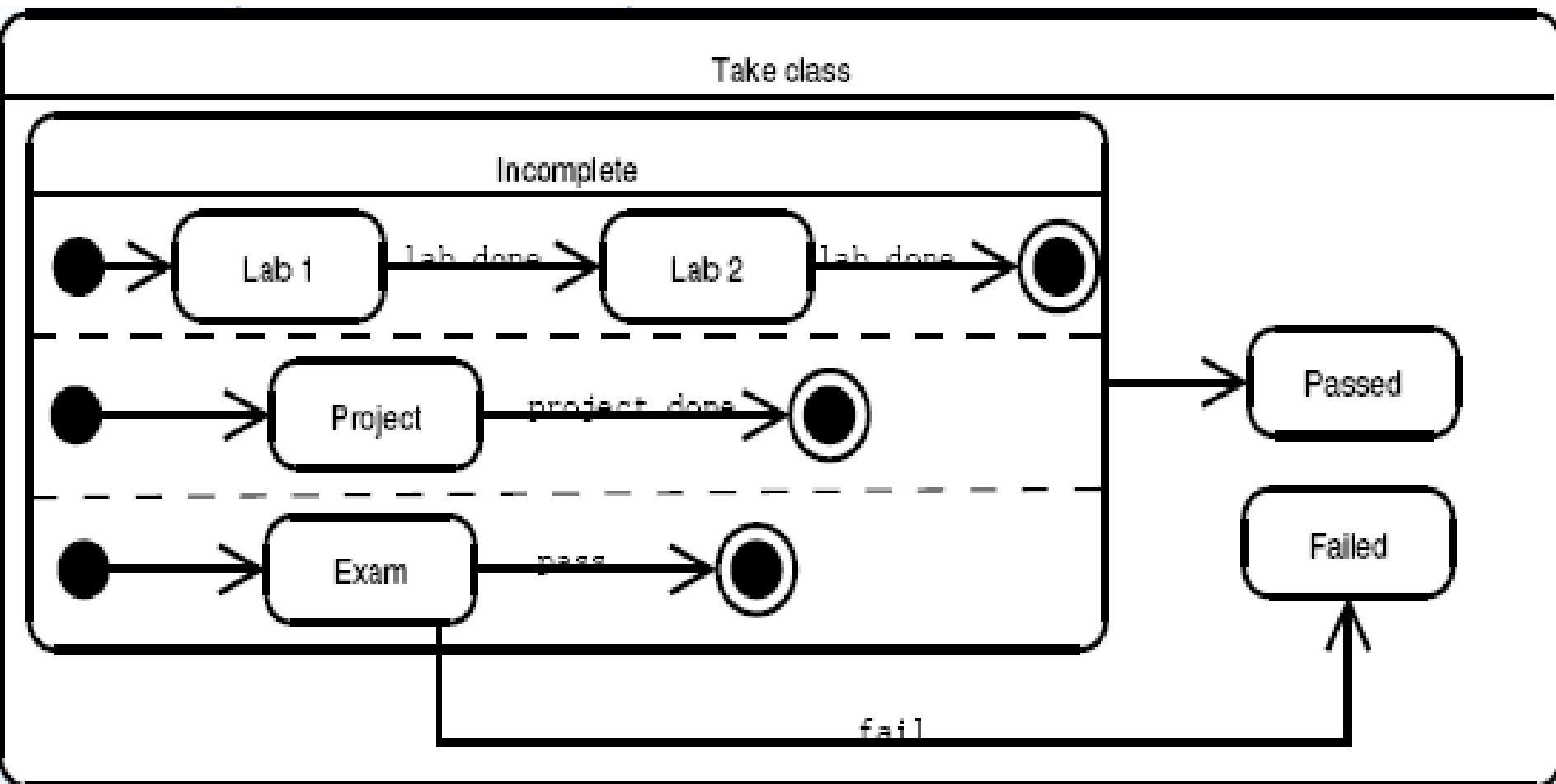
Exemplu de Stare compusă 1

- ▶ Stare compusă cu substări **secvențial** active:



Exemplu de Stare compusă 2

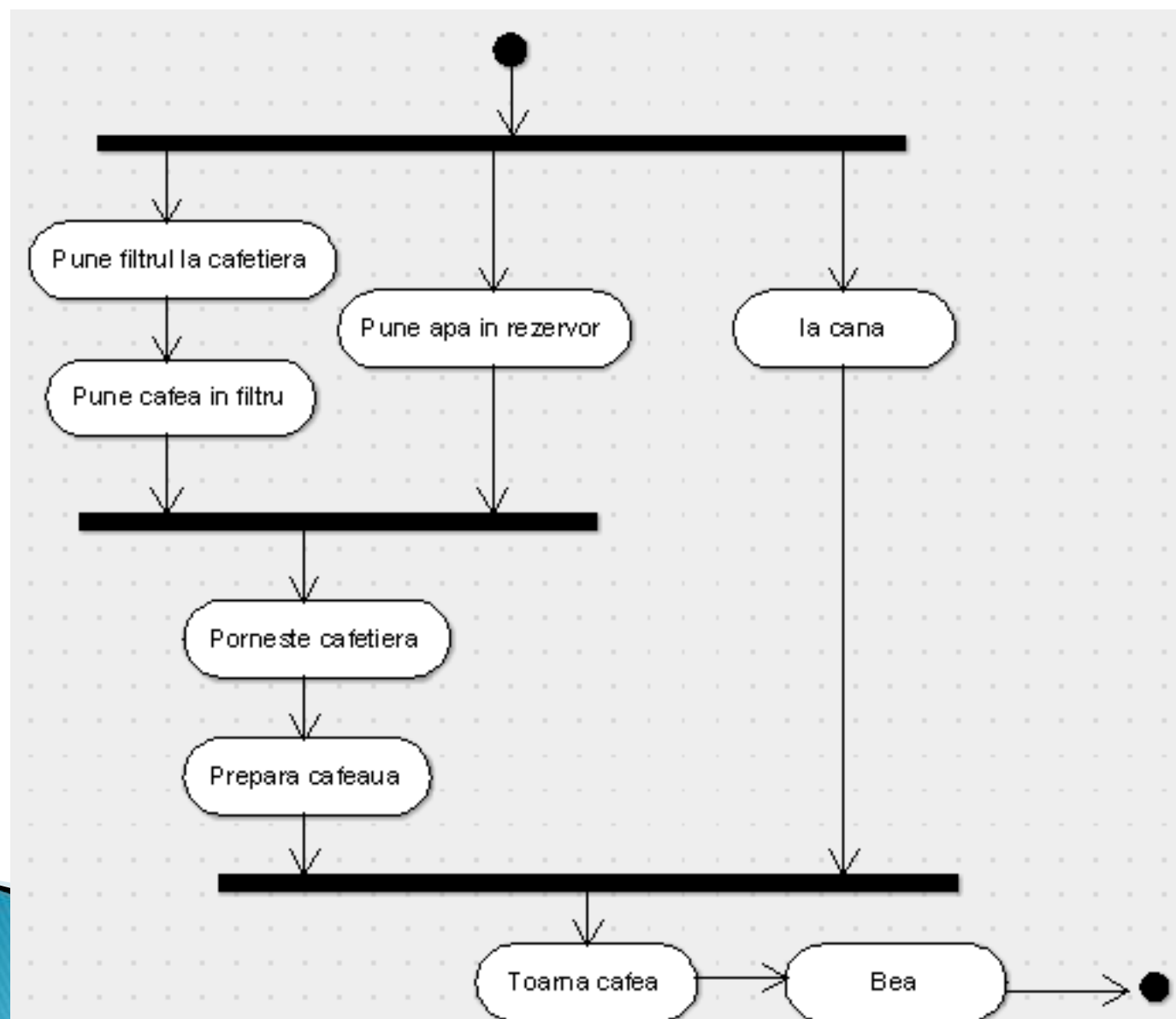
- ▶ Stare compusă cu substări **paralele** active:

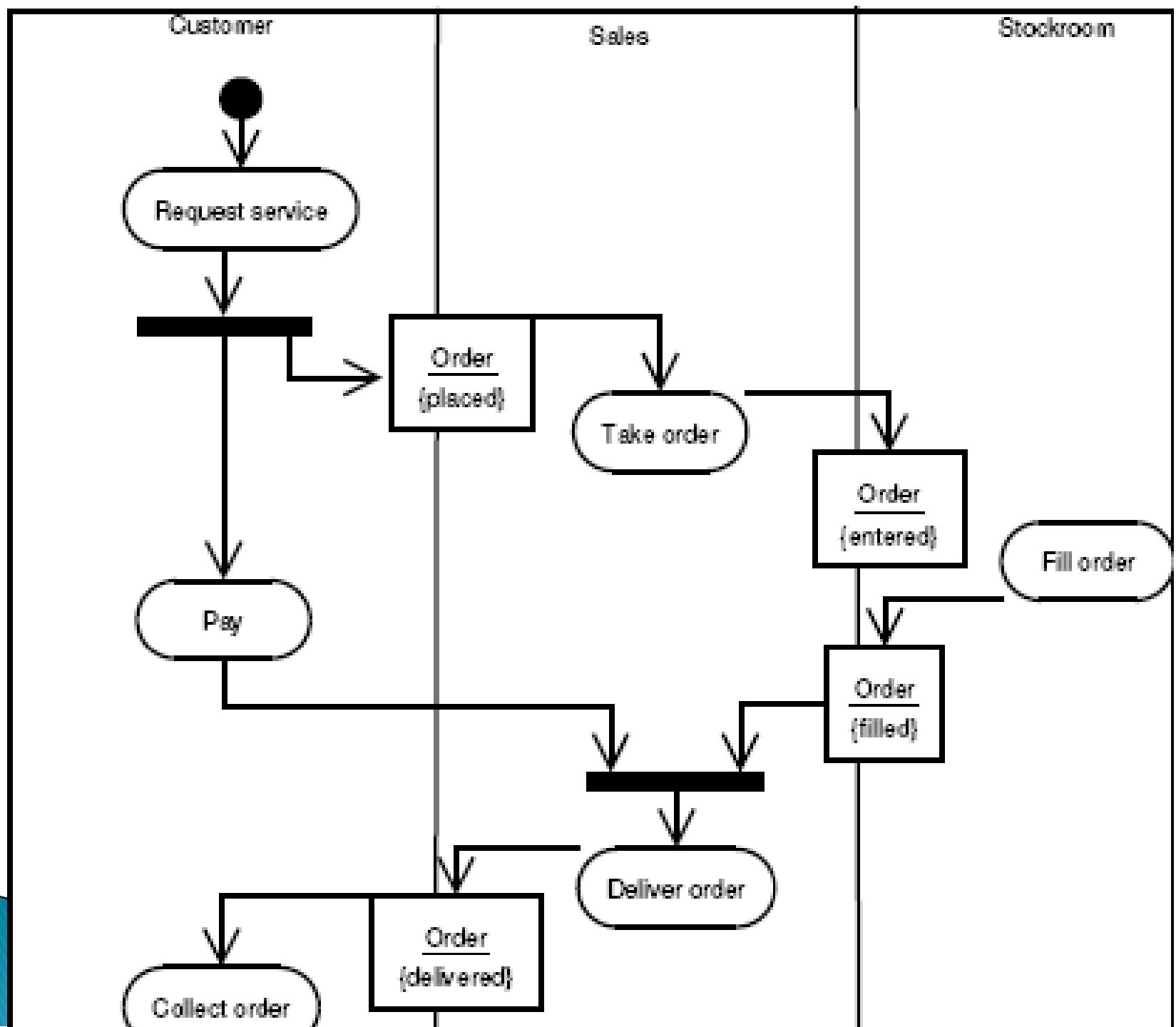


Diagramă de Activități (Activity Diagram)

- ▶ Folosită pentru a modela dinamica unui proces sau a unei operații
- ▶ Evidențiază controlul execuției de la o activitate la alta
- ▶ Se atașează:
 - Unei clase (modelează un caz de utilizare)
 - Unui pachet
 - Implementării unei operații

Exemplu de DA (Sincronizare)



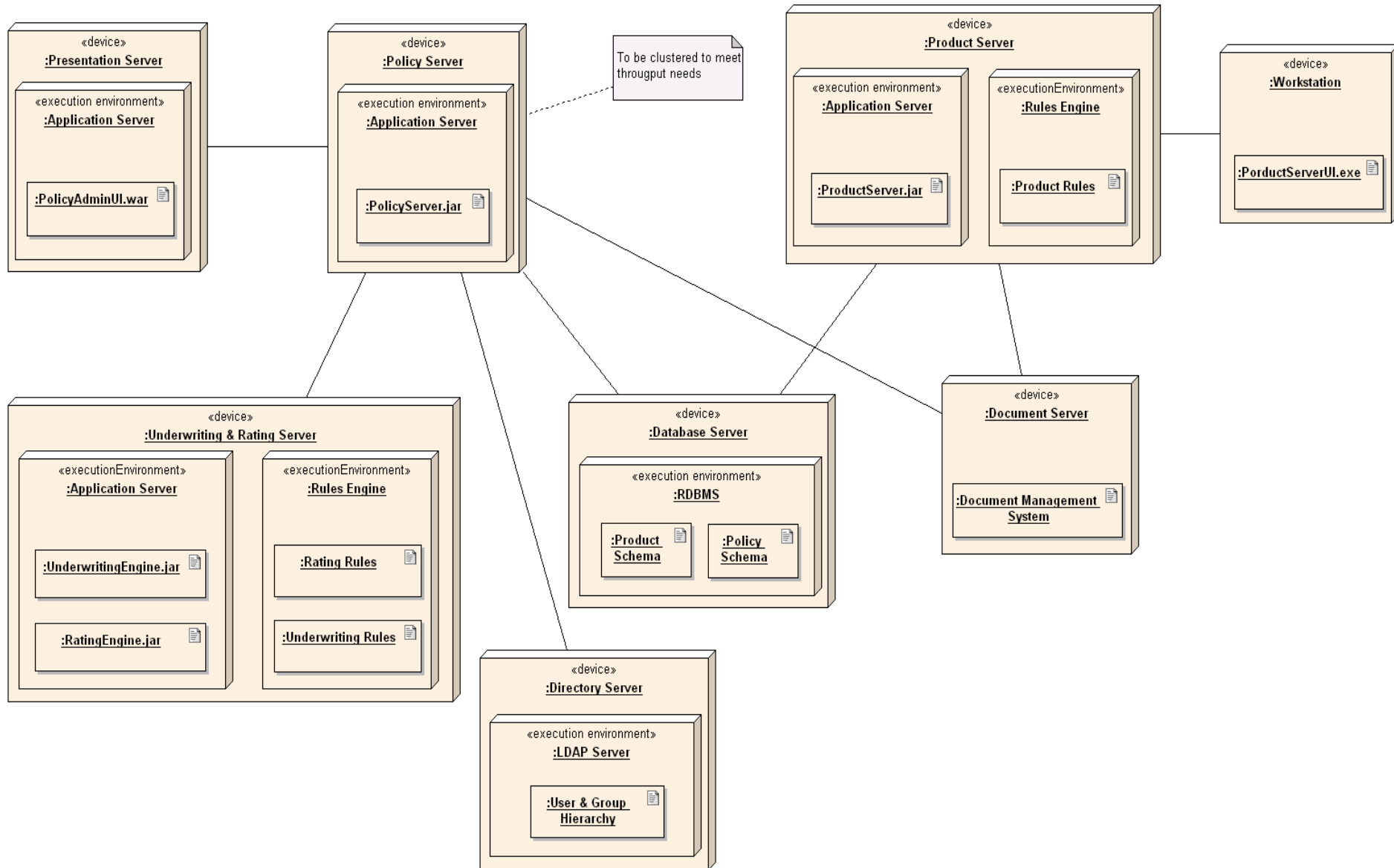


Diagrame de deployment

- ▶ Modelează mediul hardware în care va funcționa proiectul
- ▶ Exemplu: pentru a descrie un site web o diagramă de deployment va conține **componentele hardware**
 - server-ul web,
 - server-ul de aplicații,
 - server-ul de baze de date
- ▶ **Componentele software** de pe fiecare din acestea
 - Aplicația web
 - Baza de date
- ▶ Modul în care acestea sunt conectate:
 - JDBC, REST, RMI

Diagramă de deployment – Exemplu 1

dd Deployment of Components



Diagrame de Pachete (Package Diagram)

► Pachetul:

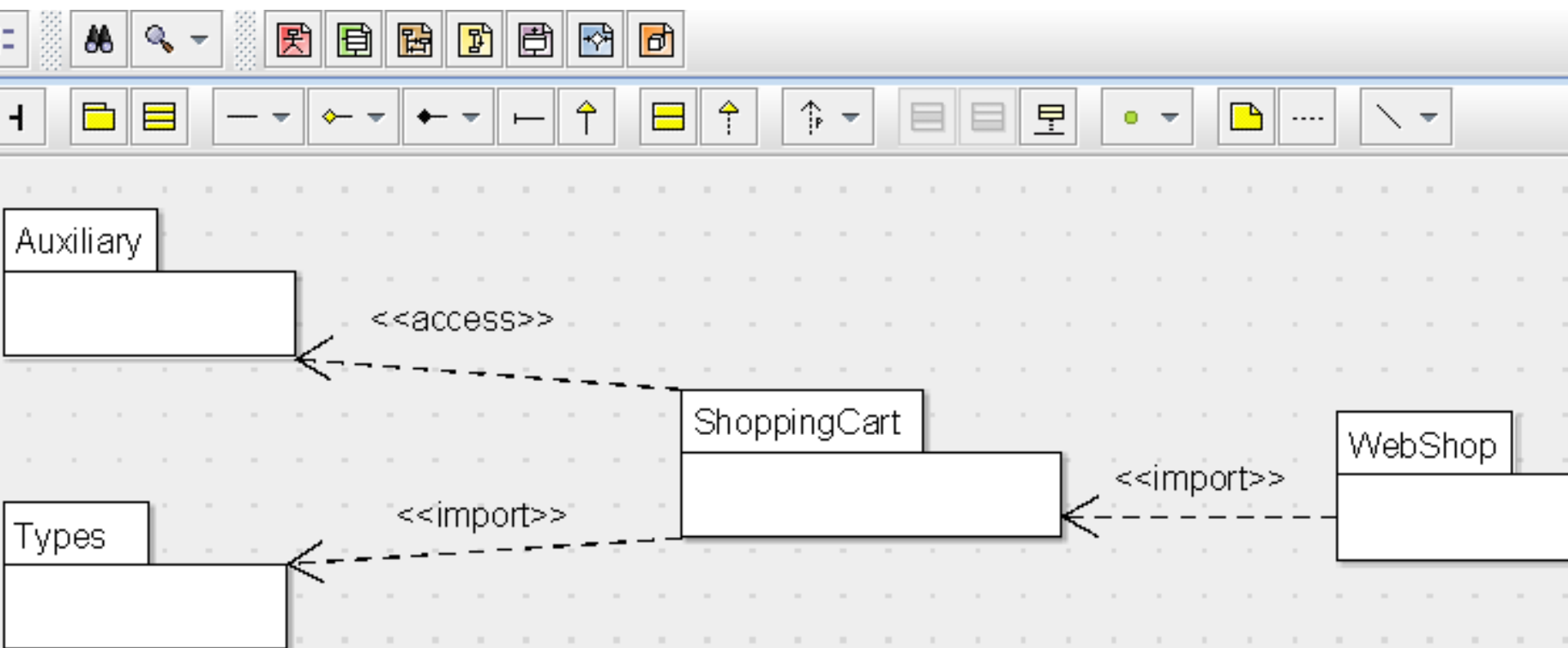
- Este un container logic pentru elemente între care se stabilesc legături
- Definește un spațiu de nume
- Toate elementele UML pot fi grupate în pachete (cel mai des pachetele sunt folosite pentru a grupa clase)
- Un pachet poate conține subpachete => se creează o structură arborescentă (similară cu organizarea fișierele / directoarelor)

Diagrame de Pachete 2

- ▶ Relații:
 - dependență `<<access>>` = import privat
 - dependență `<<import>>` = import public
- ▶ Ambele relații permit folosirea elementelor aflate în pachetul destinație de către elementele aflate în pachetul sursă fără a fi necesară calificarea numelor elementelor din pachetul destinație (similar directivei **import** din java)
- ▶ Aceste tipuri de diagrame se realizează în cadrul diagramelor de clasă

Exemplu de Diagramă de Pachete

- ▶ Elementele din Types sunt importate în ShoppingCart și apoi sunt importate mai departe de către WebShop
- ▶ Elementele din Auxiliary pot fi accesate însă doar din ShoppingCart și nu pot fi referite folosind nume necalificate din WebShop



Utilitatea diagramelor de pachete

- ▶ Împart sisteme mari în subsisteme mai mici și mai ușor de gestionat
- ▶ Permit dezvoltare paralelă iterativă
- ▶ Definirea unor interfețe clare între pachete promovează refolosirea codului (ex. pachet care oferă funcții grafice, pachet care oferă posibilitatea conectării la BD, etc...)

Recomandări în realizarea diagramelor UML

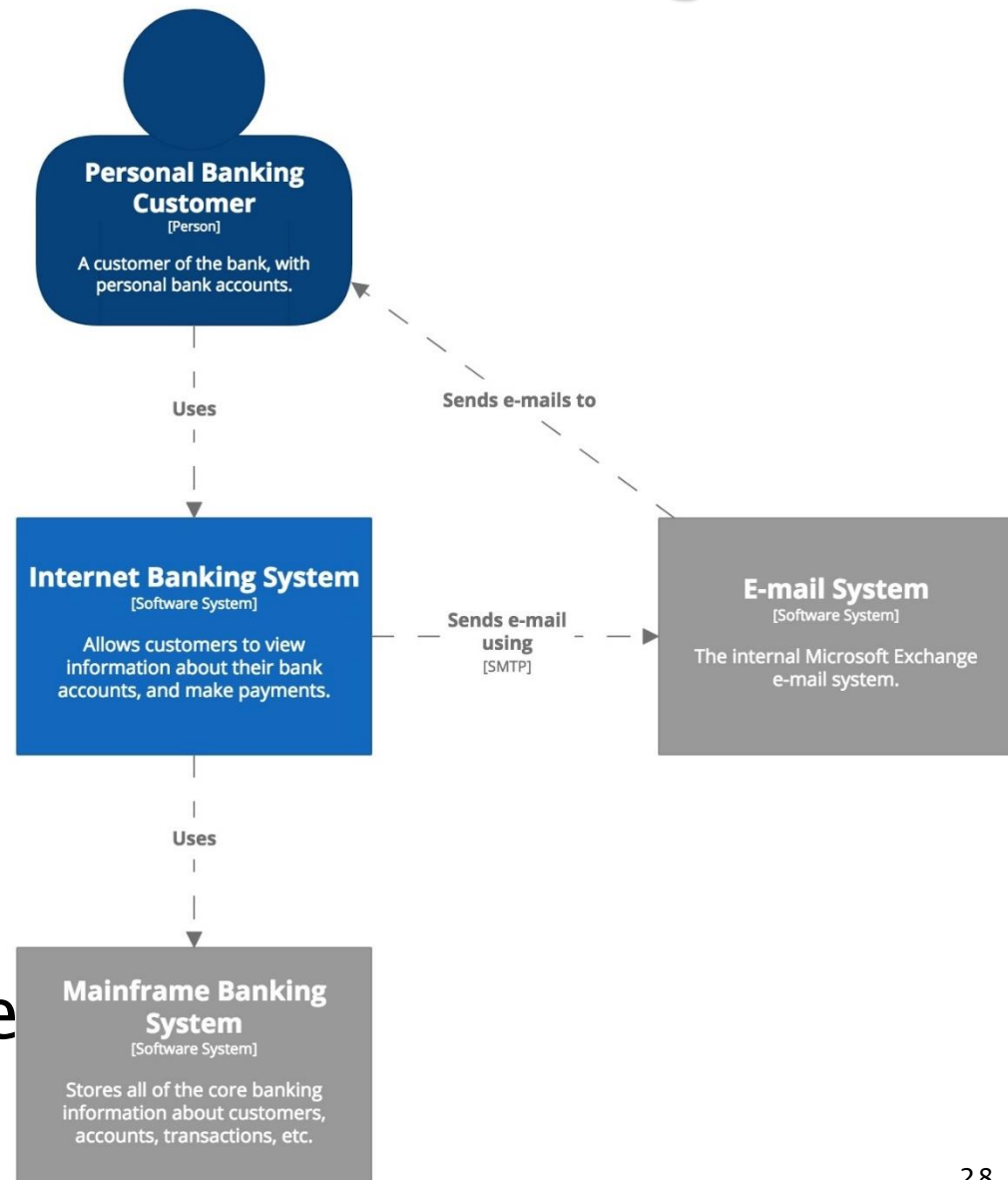
- ▶ Diagramele să nu fie nici prea complicate, dar nici prea simple: scopul este comunicarea eficientă
- ▶ Dați nume sugestive elementelor componente
- ▶ Aranjați elementele astfel încât liniile să nu se intersecteze
- ▶ Încercați să nu arătați prea multe tipuri de relații odată (evitați diagramele foarte complicate)
- ▶ Dacă este nevoie, realizați mai multe diagrame de același tip

The C4 Model for Software Architecture

- ▶ **Context, Containers, Components and Code**
- ▶ **Provides different levels of abstraction, each of which is relevant to a different audience**
- ▶ **Avoid ambiguity in your diagrams by including a sufficient amount of text as well as a key/legend for the notation you use**

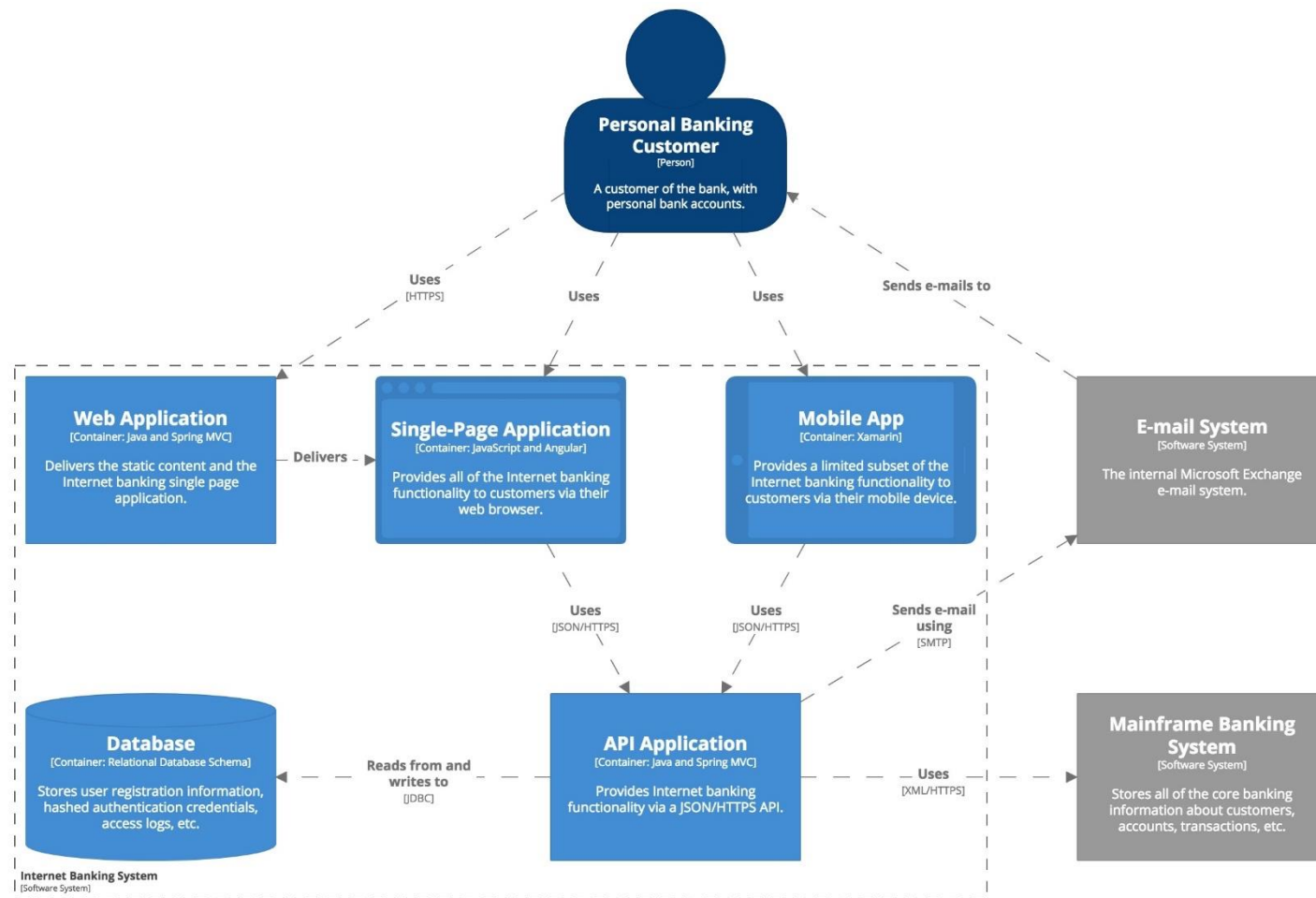
C4 Model – Level 1: System context diagram

- ▶ Shows the software system you are building
- ▶ How it fits into the world in terms of the people who use it and the other software systems it interacts with
- ▶ Colours – Systems already exist (the grey boxes) and those to be built (blue)



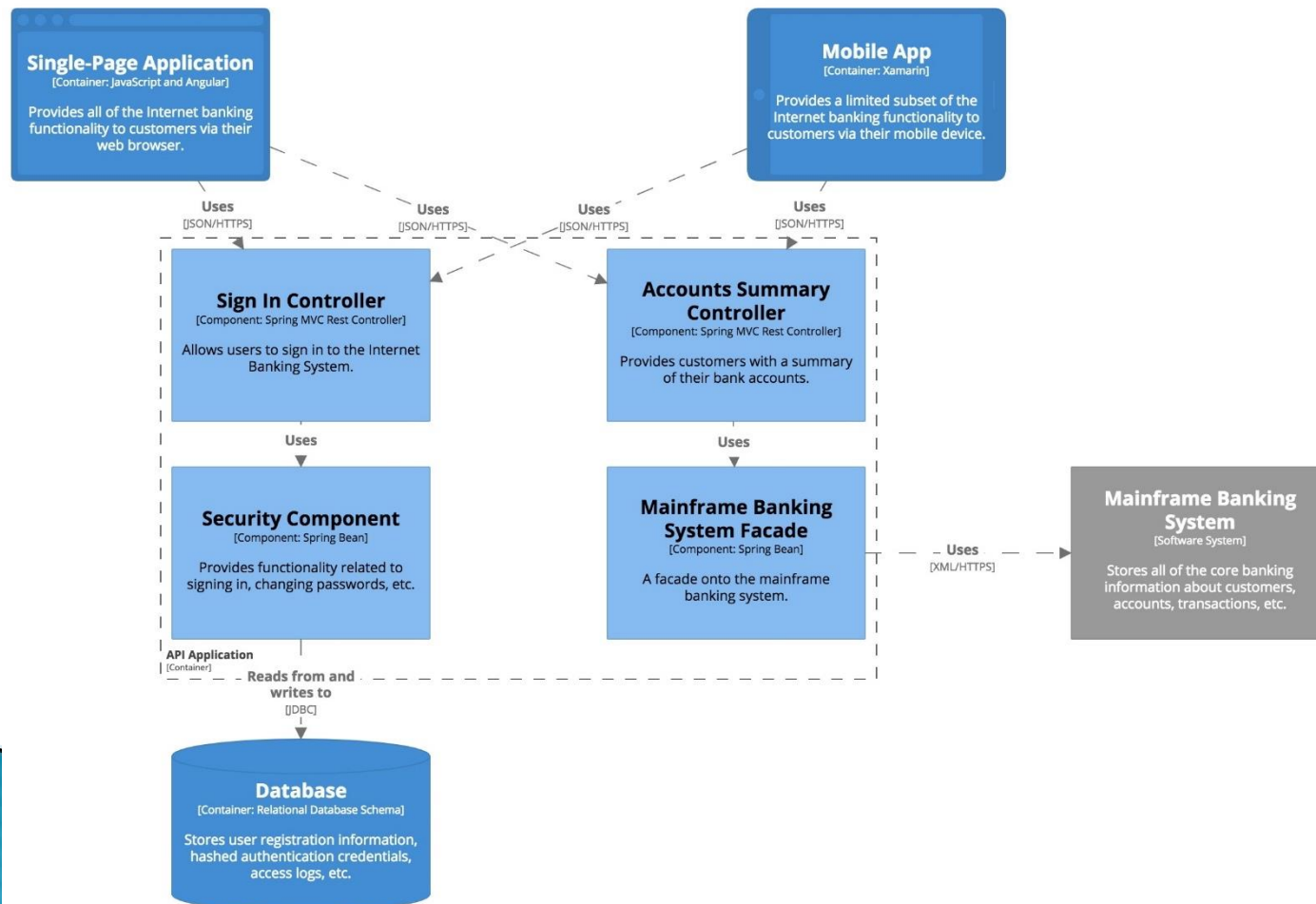
C4 Model – Level 2: Container diagram

- Zooms into the software system, and shows the containers (applications, data stores, microservices, etc.)



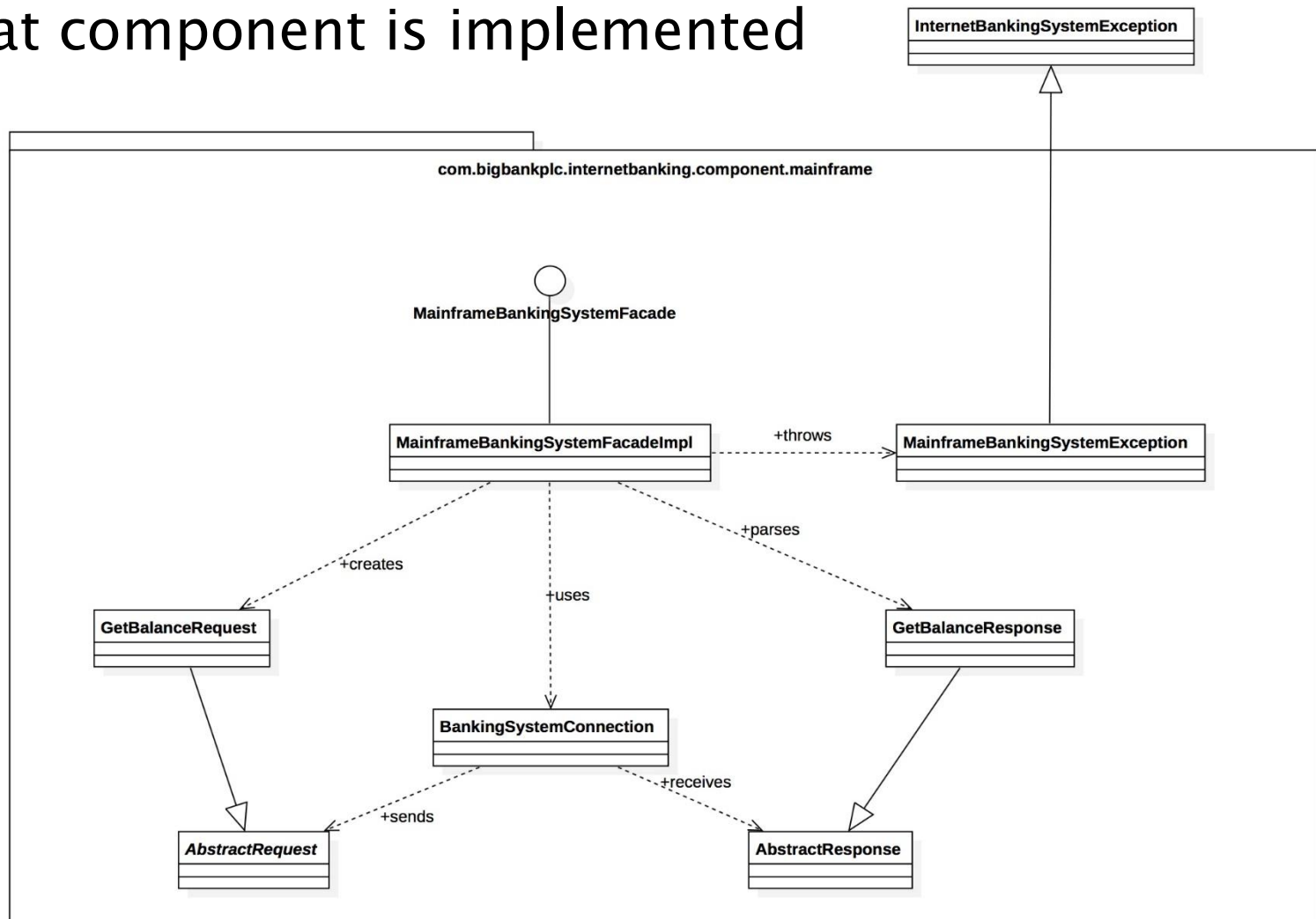
C4 Model – Level 3: Component diagram

- Zooms into an individual container to show the components inside it



C4 Model – Level 4: Code

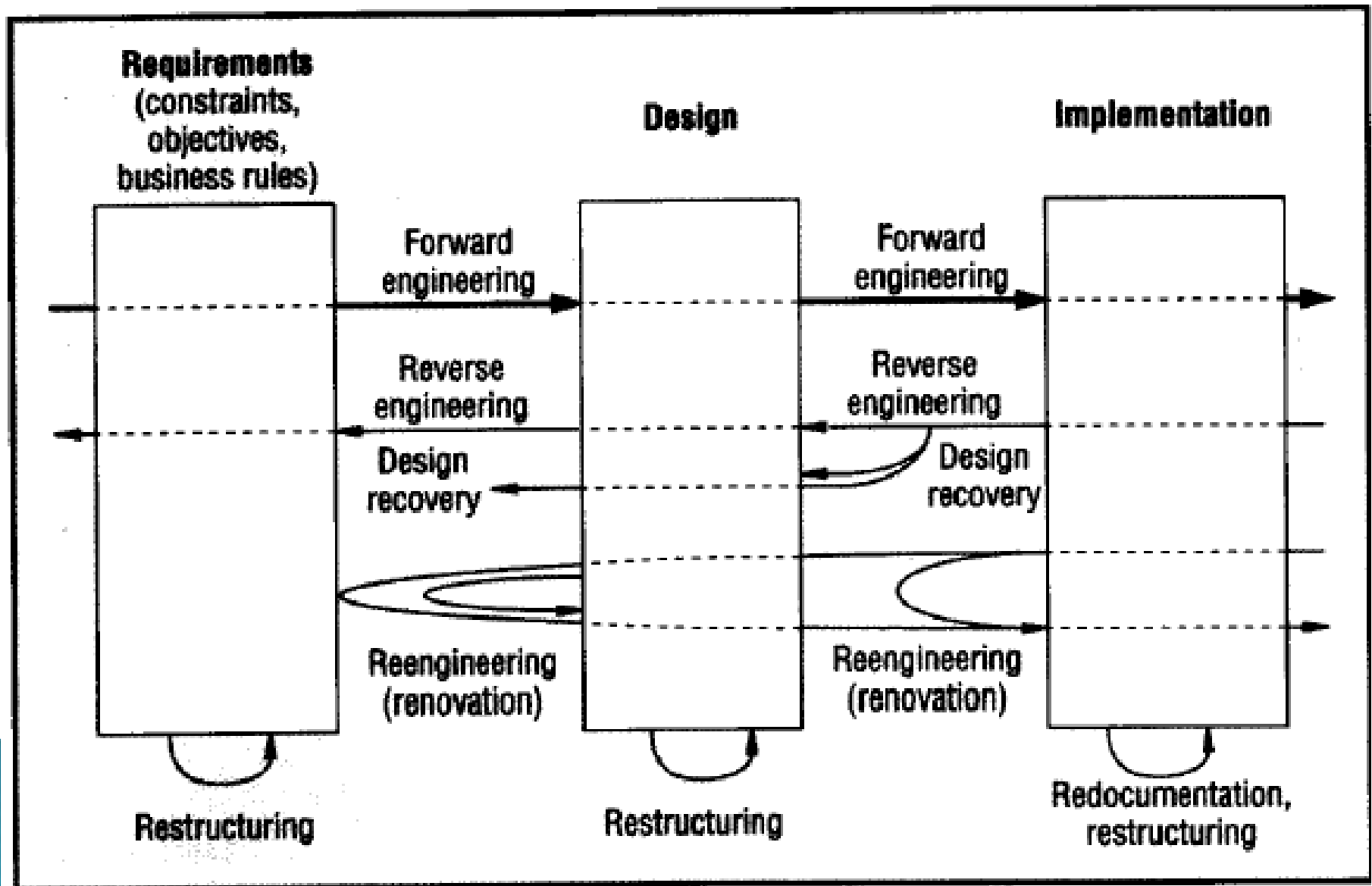
- Zoom into an individual component to show how that component is implemented



C4 Model – Links

- ▶ <https://tobiashochguertel.github.io/c4-draw.io/>
- ▶ <https://structurizr.com/express>
- ▶ <https://www.infoq.com/articles/C4-architecture-model>
- ▶ <https://c4model.com/>

Forward and Reverse Engineering



Forward Engineering

- ▶ A traditional process of moving from high-level abstractions and logical to the implementation-independent designs to the physical implementation of a system
- ▶ FE follows a sequence of going from requirements through designing its implementation

Reverse Engineering

- ▶ **Reverse engineering (RE)** is the process of discovering the technological principles of a device, object or system through analysis of its structure, function and operation
- ▶ *To try to make a new device or program that does the same thing without copying anything from the original*
- ▶ Reverse engineering has its origins in the analysis of hardware for commercial or military advantage

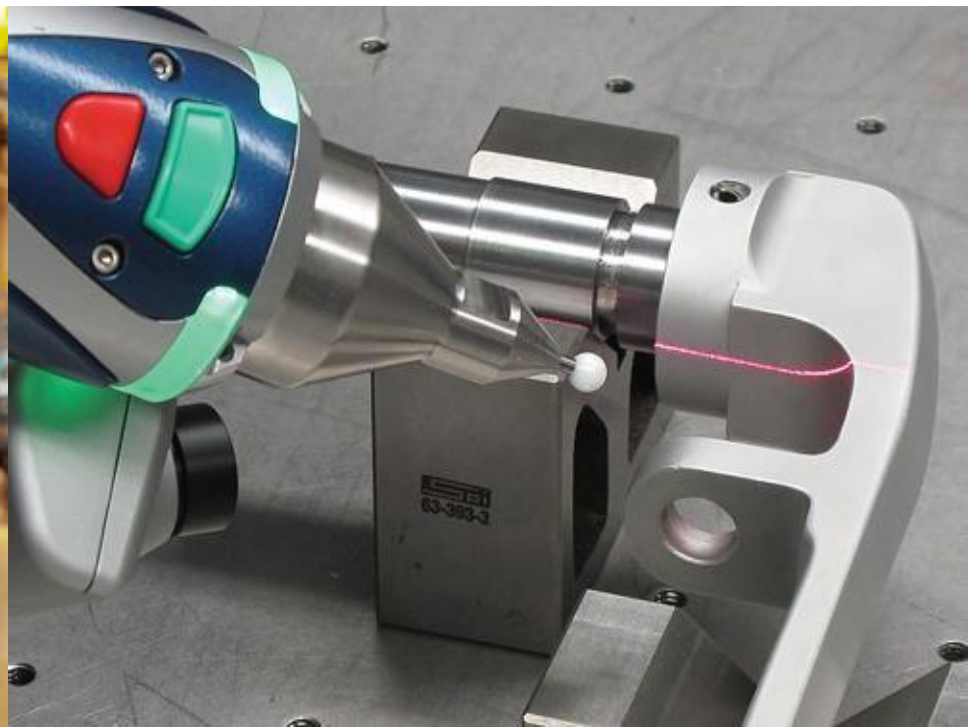
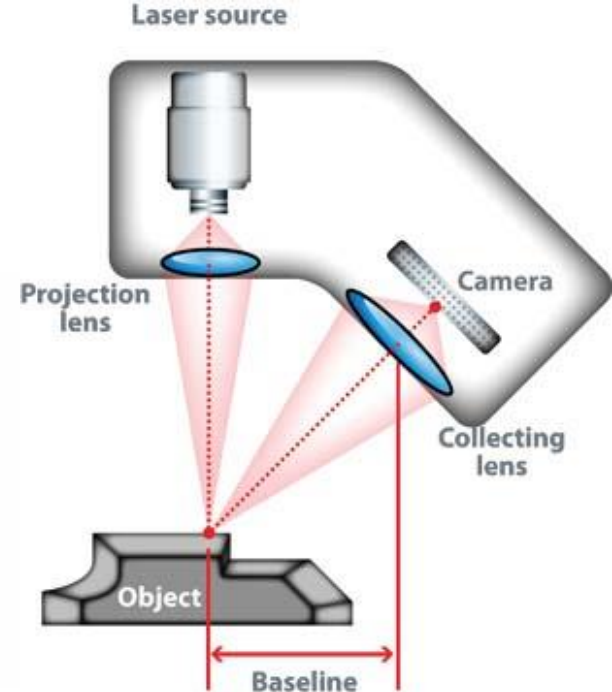
RE Motivation

- ▶ Interoperability
- ▶ Lost documentation
- ▶ Product analysis
- ▶ Security auditing
- ▶ Removal of copy protection, circumvention of access restrictions
- ▶ Creation of unlicensed/unapproved duplicates
- ▶ Academic/learning purposes
- ▶ Curiosity
- ▶ Competitive technical intelligence (understand what your competitor is actually doing versus what they say they are doing)
- ▶ Learning: Learn from others mistakes

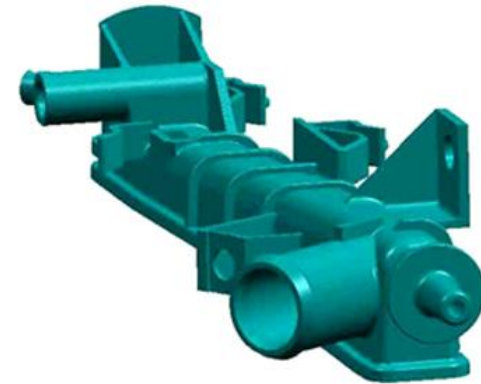
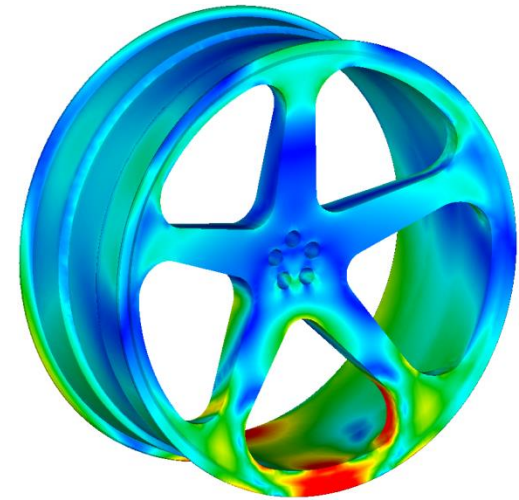
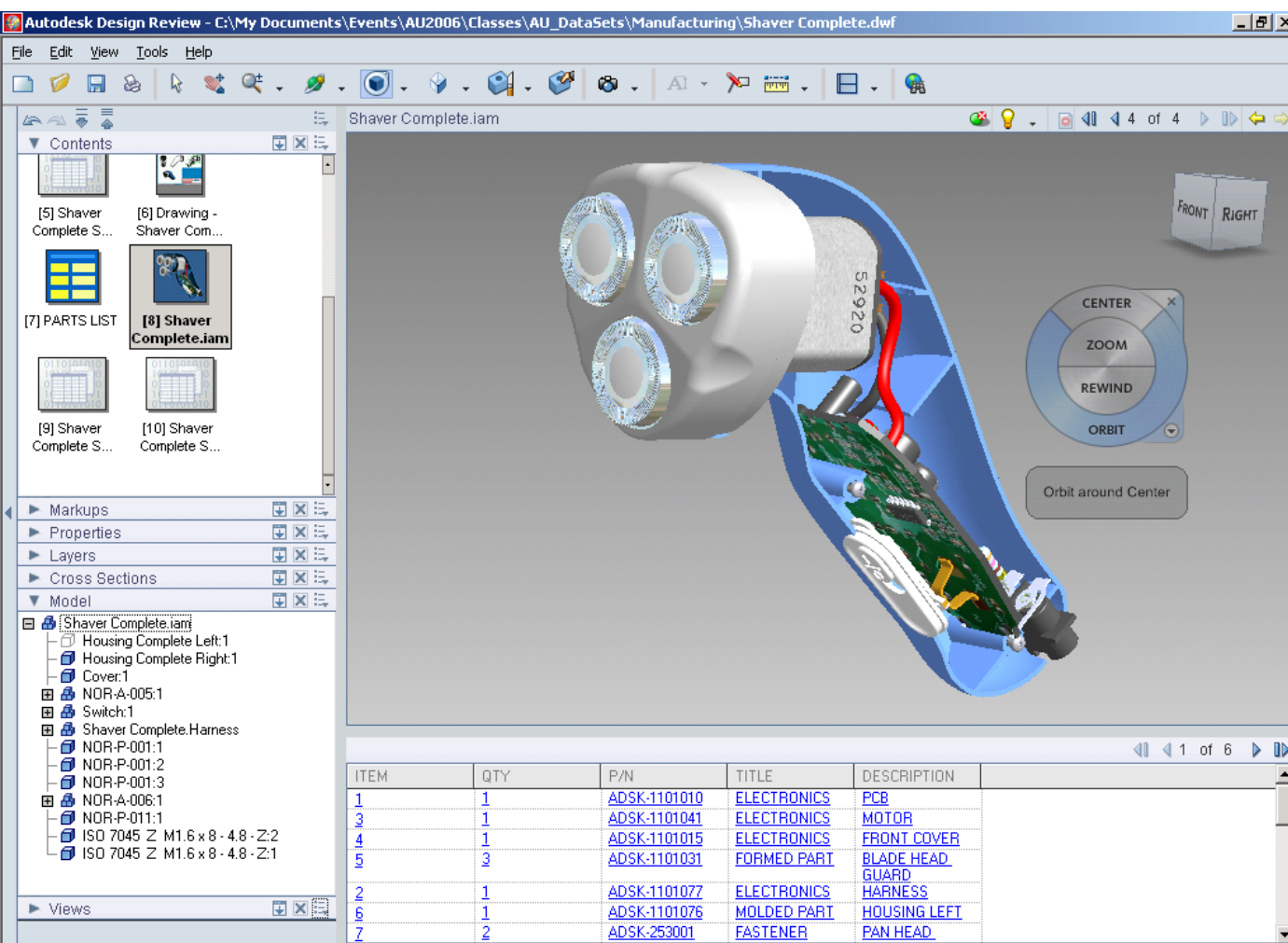
Types of RE

- ▶ RE1: Reverse engineering of mechanical devices
- ▶ RE2: Reverse engineering of integrated circuits/smart cards
- ▶ RE3: Reverse engineering for military applications
- ▶ RE4: Reverse engineering of software

RE1 : Scanere laser 3D



RE1: Servicii de modelare 3D CAD



RE1 : Servicii de imprimare 3D



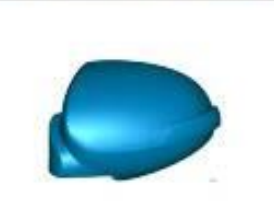
- ▶ Rapid prototyping

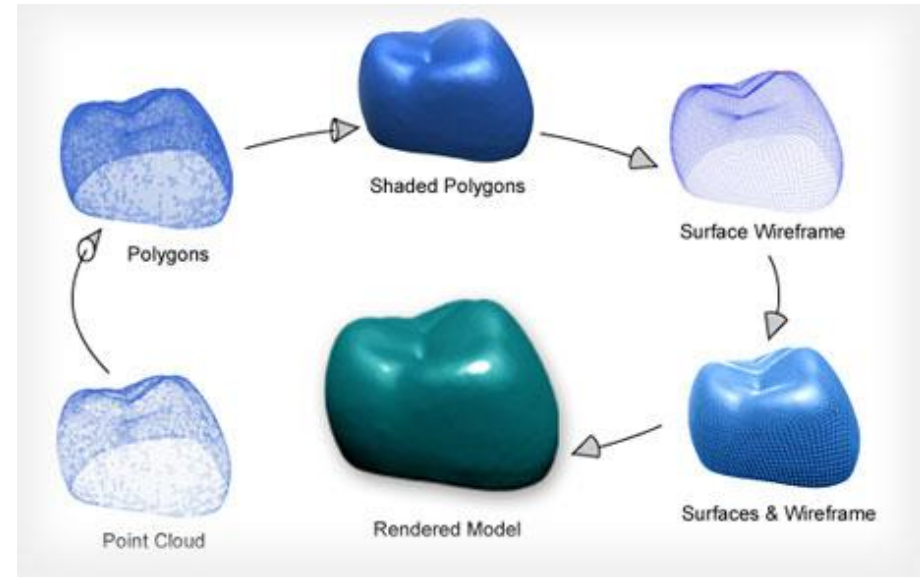


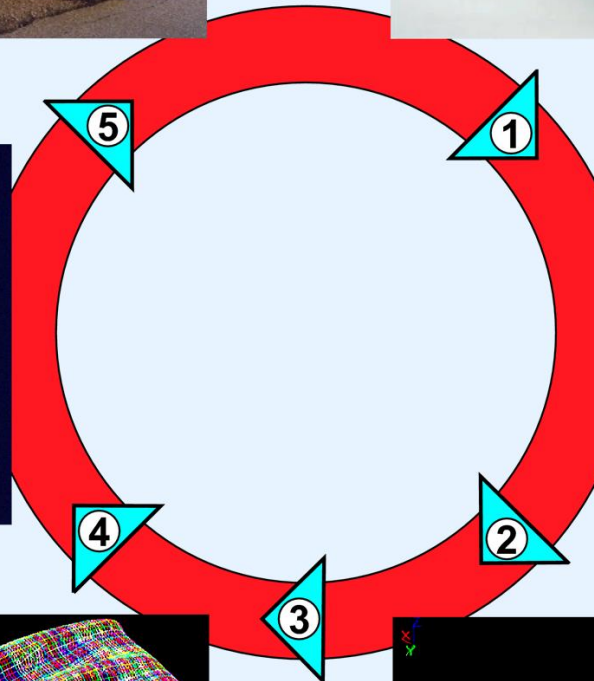
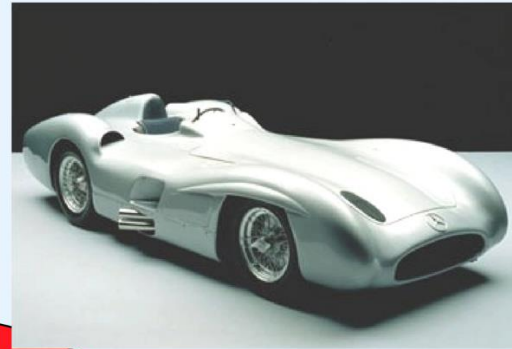
- ▶ FullCure materials



RE1 : Domenii

Physical Part	Modeling Data
	
	
	





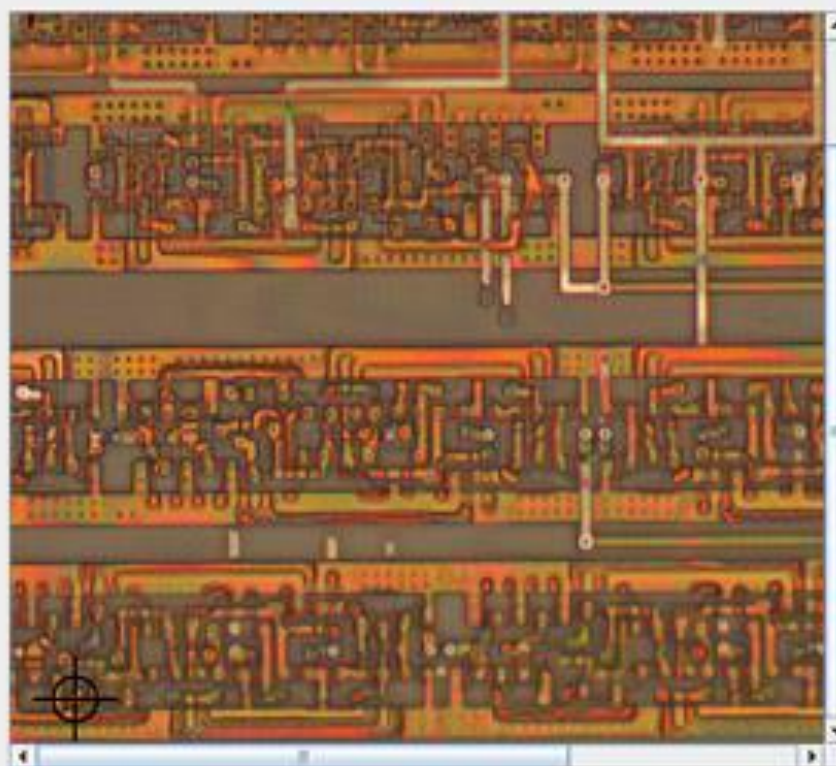
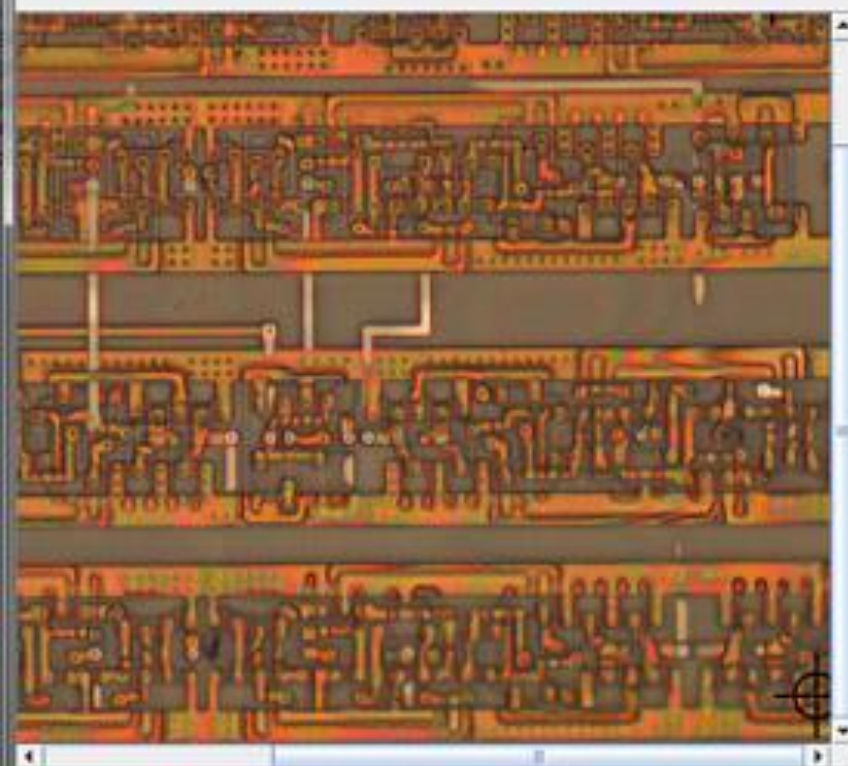
Reverse engineering of integrated circuits/smart cards

- ▶ RE is an invasive and destructive form of analyzing a smart card
- ▶ The attacker grinds away layer by layer of the smart card and takes pictures with an electron microscope
- ▶ Engineers employ sensors to detect and prevent this attack

3. Schritt: Setzen der Kontrollpunkte



☐ D:\...TFH DATEIEN ...lax Praktika\CRElaChipBilder\iz...



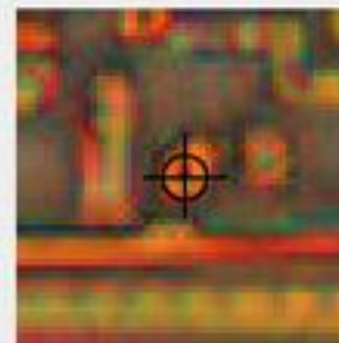
654 - 63

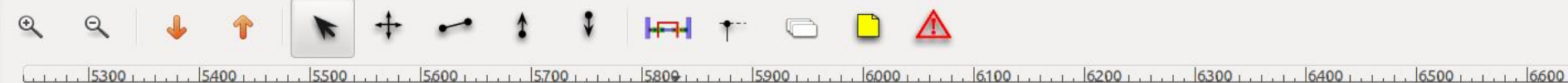
680 - 484



12 - 64

38 - 486





Logic gates

Short Name	#	Width	Height	Fill color	Frame color	Description
01-FF	58	272	134			D-Q-FlipFlop
02-FF	11	343	134			D-Q-FlipFlop with rst
03-FF	17	343	133			D-Q-FlipFlop with rts
04-BUF	5	226	128			
05-3XOR	13	249	133			
06: 2-XNOR	12	133	133			

Edit Add Remove Close

Edit gate

Entity Behaviour Layout

Short name: 02-FF

Description: D-Q-FlipFlop with rst

Logic Class: flipflop (generic)

Port ID	Port Name	Port Description	In	Out
2384	!Q	!Q	☐	☒
2380	Q	Q	☐	☒
2388	clk	clk	☒	☐
2386	D	D	☒	☐
2382	rst	rst	☒	☐

Ports:

Add Remove

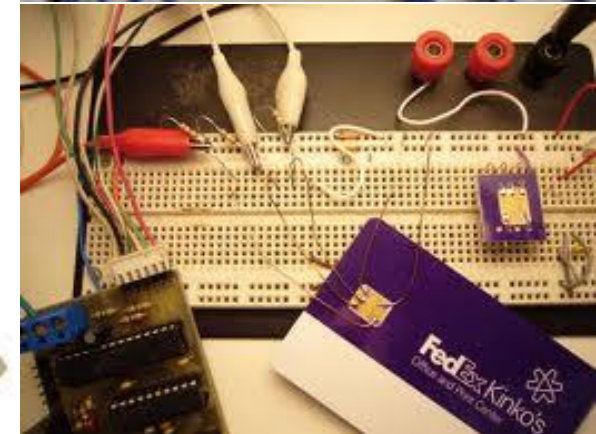
Fill color: Reset Color

Frame color: Reset Color

Cancel OK

RE2: Smart cards

- ▶ Satellite TV
- ▶ Security card
- ▶ Phone card
- ▶ Ticket card
- ▶ Bank card



Reverse engineering for military applications

- ▶ Reverse engineering is often used by militaries in order to copy other nations' technologies, devices or information that have been obtained by regular troops in the fields or by intelligence operations
- ▶ It was often used during the Second World War and the Cold War
- ▶ Well-known examples from WWII and later include: rocket, missile, bombers, China has reversed many examples of US and Russian hardware, from fighter aircraft to missiles and HMMWV cars

RE3: Avioane

► US – B-29

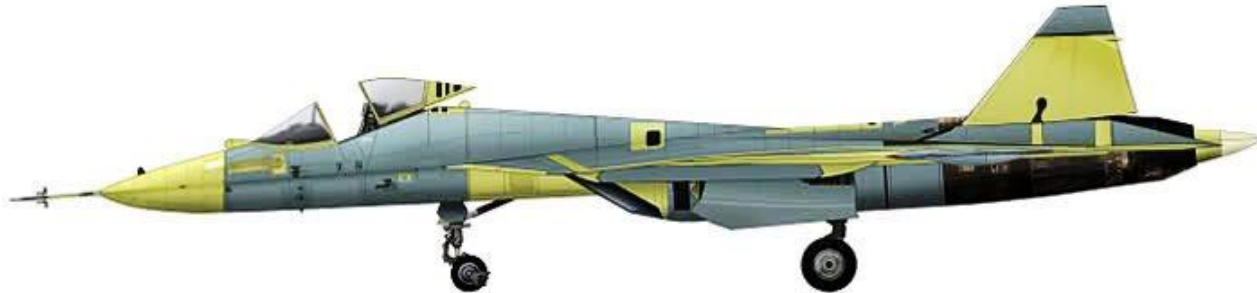
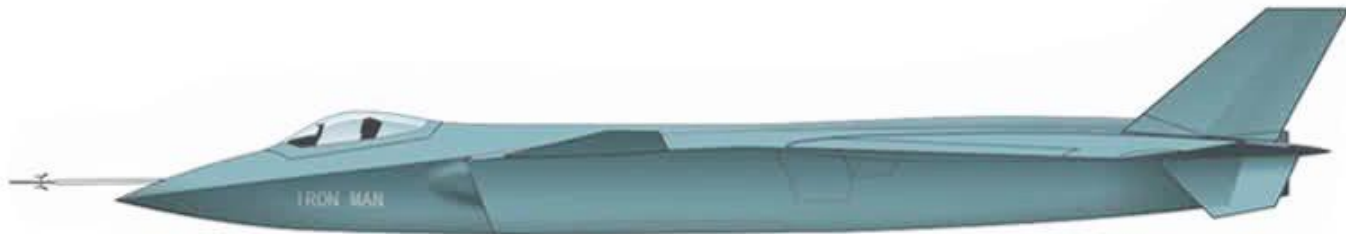


URSS – Tupolev Tu-4



RE3: Avioane (2)

- ▶ Chinese J-20, Black Eagle US F-22, Russian Sukhoi T-50



RE3: Rachete

- ▶ US – AIM-9 Sidewinder Soviet – Vympel K-13



Russia's new ballistic missile submarine

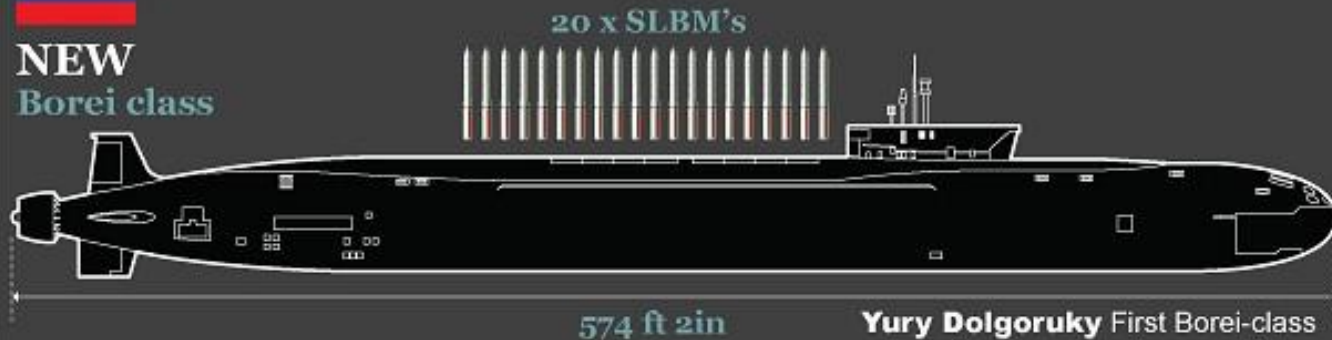
RE3: Submar



OLD
Typhoon class



NEW
Borei class



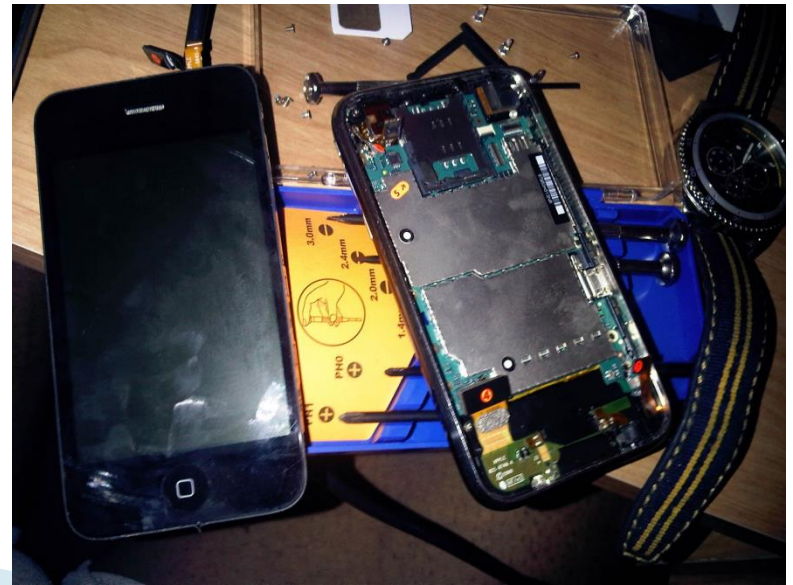
Royal Navy
Vanguard class



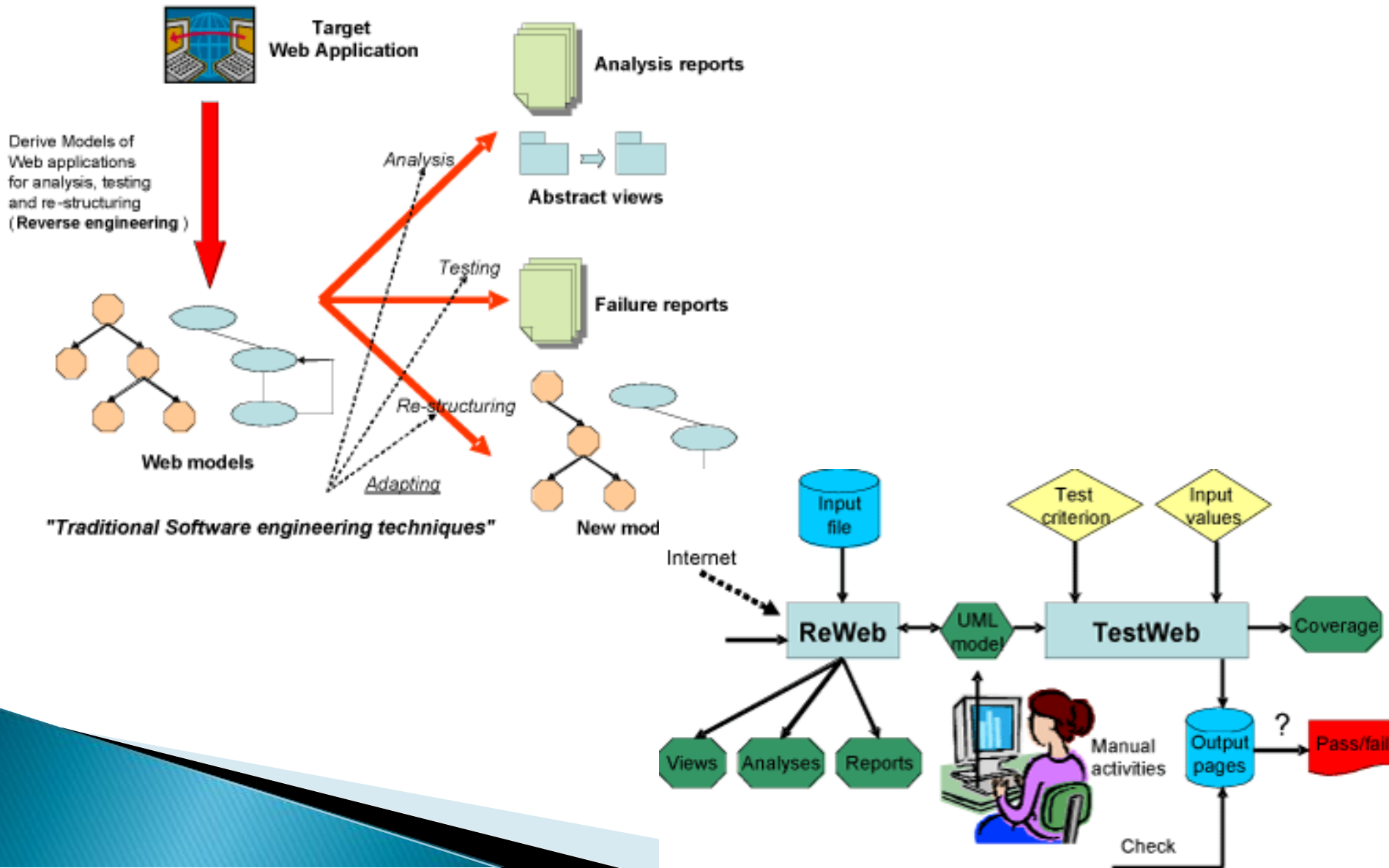
Reverse engineering of software

- ▶ Reverse engineering is the process of analyzing a subject system to create representations of the system at a higher level of abstraction
- ▶ In practice, two main types of RE emerge:
 - Source code is available (but it is poorly documented)
 - There is no source code available for the software
- ▶ **Black box testing** in software engineering has a lot in common with reverse engineering

RE4: Smart phones



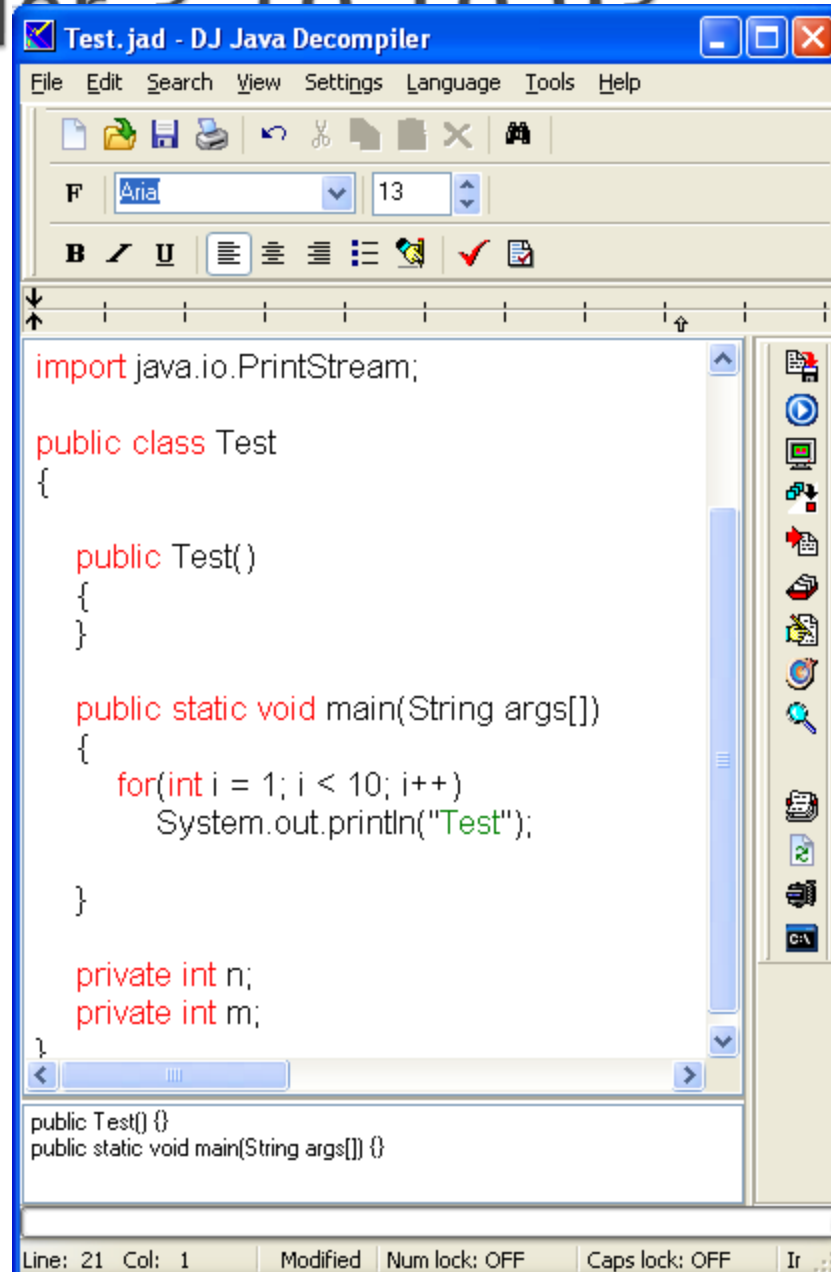
RE4 of Web Applications



RE4: DJ Java Decompiler 2.10.10.02

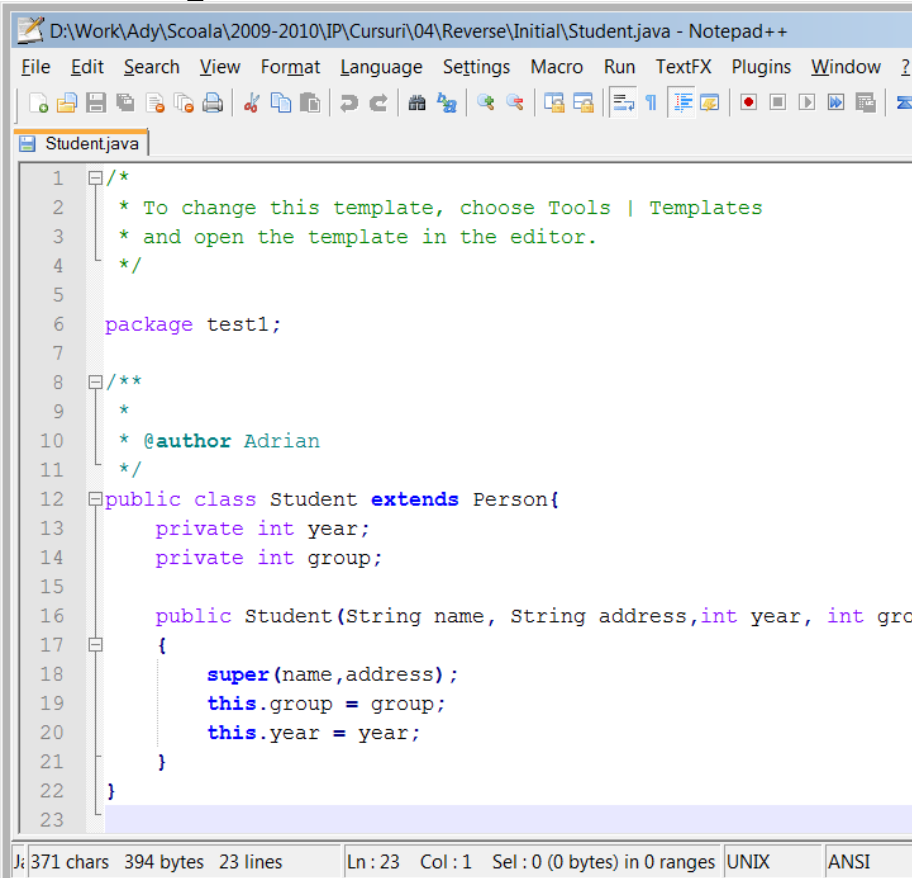
```
public class Test
{
    private int n;
    private int m;

    public static void main(String
args[])
    {
        for(int i=1;i<10;i++)
            System.out.println("Test");
    }
}
```



RE4: JAD

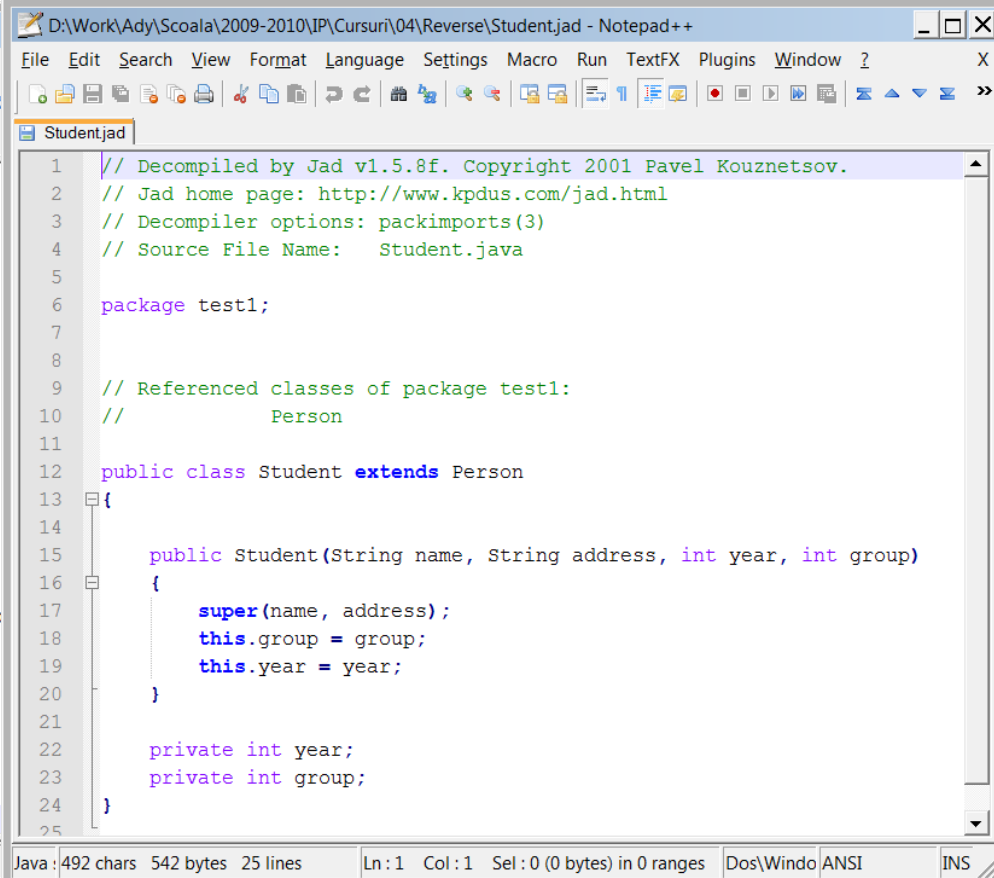
- ▶ Link: <http://www.steike.com/code/java-reverse-engineering/>
- ▶ `jad.exe NumeFisier.class => NumeFisier.jad`



D:\Work\Ady\Scoala\2009-2010\IP\Cursuri\04\Reverse\Initial\Student.java - Notepad++

```
1  /*
2  * To change this template, choose Tools | Templates
3  * and open the template in the editor.
4  */
5
6  package test1;
7
8  /**
9   *
10  * @author Adrian
11  */
12  public class Student extends Person{
13      private int year;
14      private int group;
15
16      public Student(String name, String address,int year, int gro
17      {
18          super(name,address);
19          this.group = group;
20          this.year = year;
21      }
22  }
23
```

J: 371 chars 394 bytes 23 lines Ln: 23 Col: 1 Sel: 0 (0 bytes) in 0 ranges UNIX ANSI



D:\Work\Ady\Scoala\2009-2010\IP\Cursuri\04\Reverse\Student.jad - Notepad++

```
1  // Decompiled by Jad v1.5.8f. Copyright 2001 Pavel Kouznetsov.
2  // Jad home page: http://www.kpdus.com/jad.html
3  // Decompiler options: packimports(3)
4  // Source File Name:   Student.java
5
6  package test1;
7
8
9  // Referenced classes of package test1:
10 //     Person
11
12  public class Student extends Person
13  {
14
15      public Student(String name, String address, int year, int group)
16      {
17          super(name, address);
18          this.group = group;
19          this.year = year;
20      }
21
22      private int year;
23      private int group;
24  }
25
```

Java: 492 chars 542 bytes 25 lines Ln: 1 Col: 1 Sel: 0 (0 bytes) in 0 ranges Dos\Windo ANSI INS

RE4: Open Office

ss - OpenOffice.org

File Edit View Insert Format Tools Slide Show Window Help

File Edit View Insert Format Tools Slide Show Window Help

Slide Slide Design

Normal Outline Notes Handout Slide Sorter

Slide 1

Exemplu OpenOffice.org

- Alt exemplu
- Și altul
- Și încă unul
- Și încă unul

Tasks

Master Pages

Layouts

Custom Animation

Slide Transition

Sheet 1 / 3

29,16 / -2,59 : 0,00 x 0,00 51% * Slide 1 / 1 Default

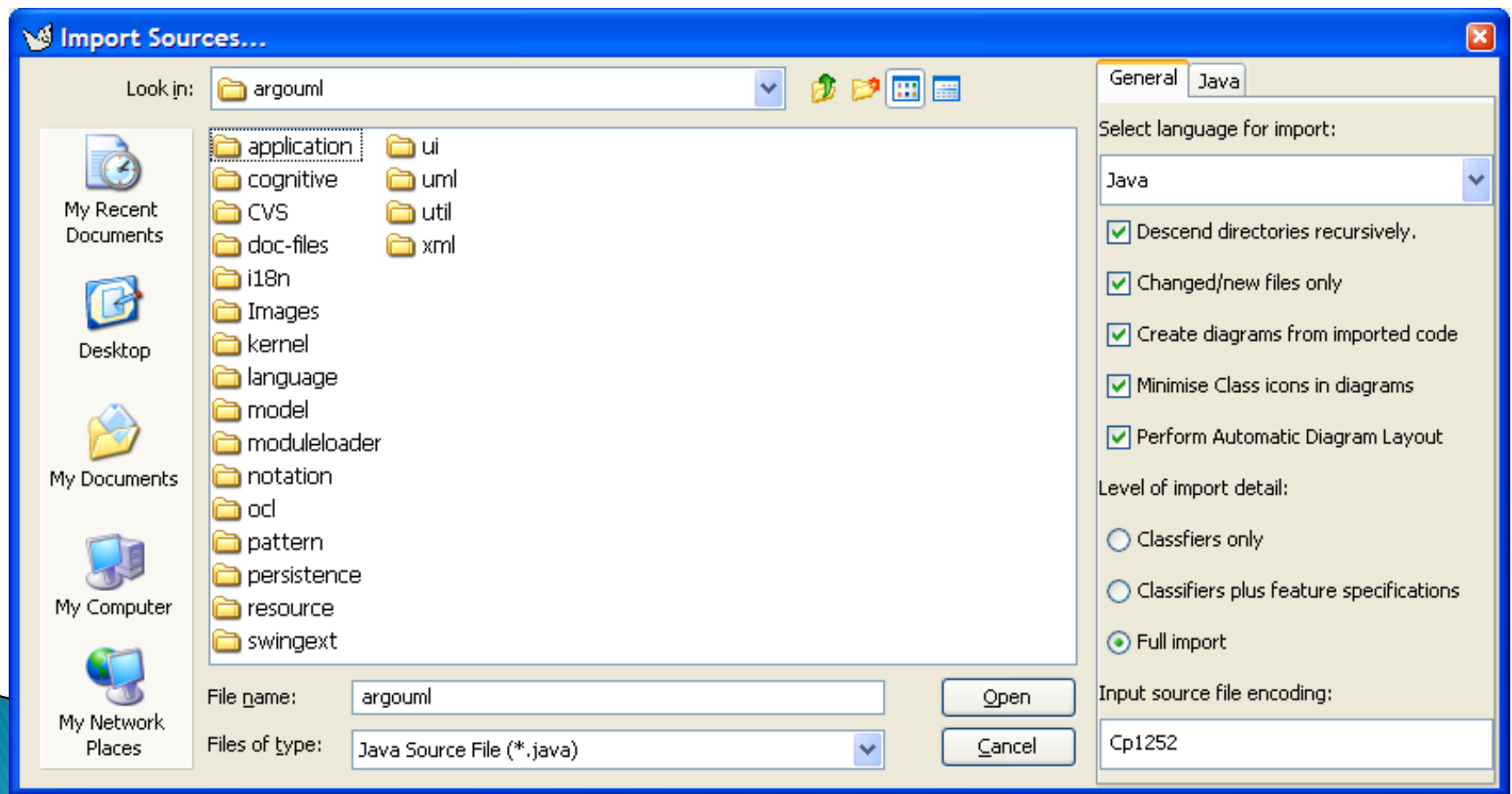
C#

The screenshot displays the Microsoft Visual Studio IDE with the following components:

- Toolbar:** Includes icons for File, Edit, View, Project, Build, Debug, Class Diagram, Data, Tools, Test, Window, and Help. The status bar shows 'Debug' and 'Any CPU'.
- Class Designer:** The main workspace shows a class diagram for 'ClassDiagram1.cd*'. It contains four classes:
 - Program** (Static Class) with a 'Main' method.
 - Form1** (Class) with a 'Form' property.
 - Settings** (Sealed Class) with an 'ApplicationSettingsBase' property.
 - Resources** (Class).
- Solution Explorer:** Shows the project structure for 'xmlView' (1 project). The files listed are 'Properties', 'References', 'ClassDiagram1.cd', and 'Form1.cs'. A context menu is open over 'Form1.cs', showing options like 'Open', 'View Code', 'View Designer', 'View Class Diagram', 'Exclude From Project', 'Cut', 'Copy', 'Delete', 'Rename', and 'Properties'.
- Error List:** Shows '0 Errors', '0 Warnings', and '0 Messages'. Below this is a table with columns: Desc..., File, Line, Column, Project.
- Build Action:** A section at the bottom right titled 'Build Action' with the text 'How the file relates to the build and deployment processes.'

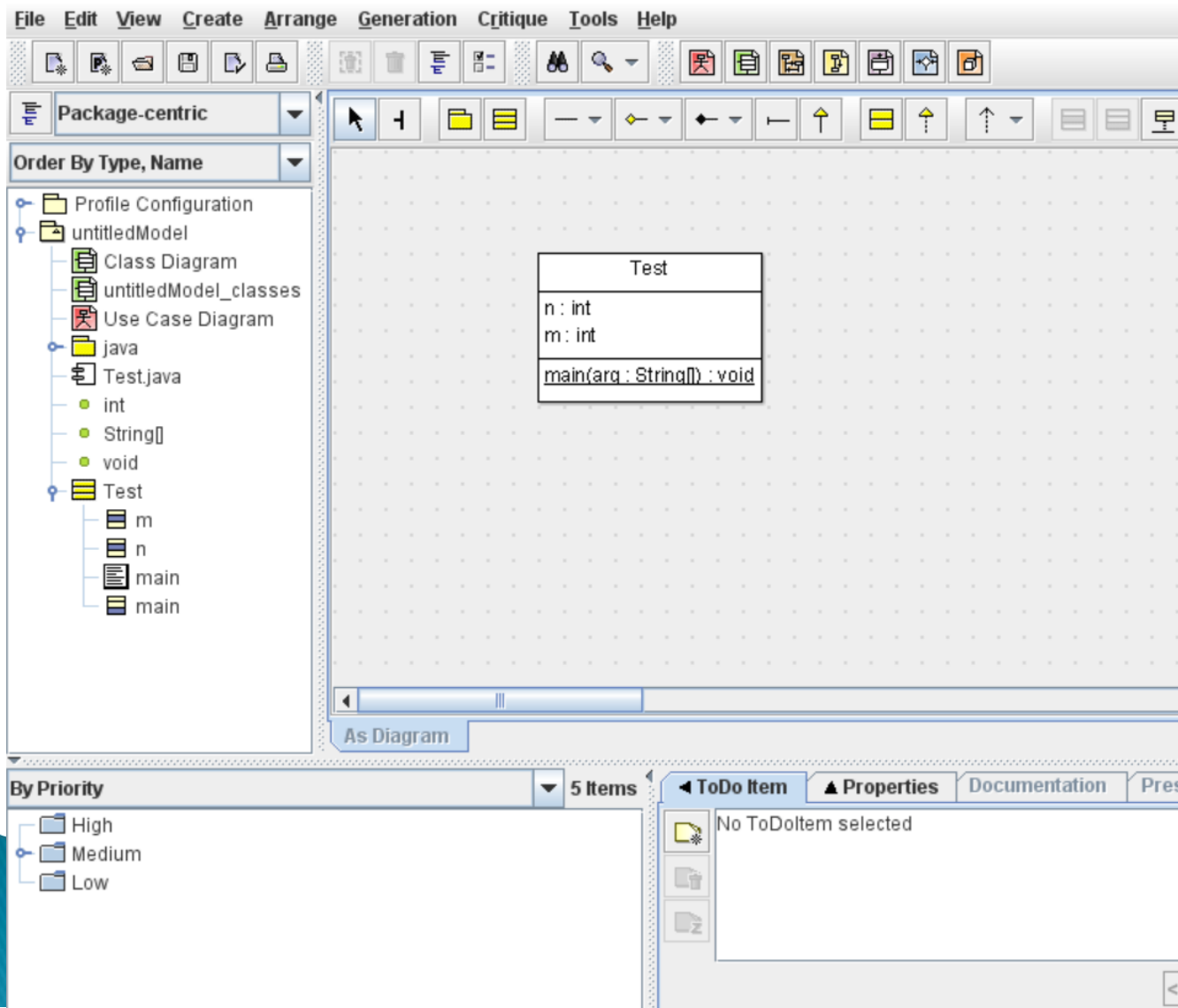
RE în ArgoUML

► File -> Import Sources...



Pentru exemplul anterior...

Untitled - Class Diagram - ArgoUML *



Using AI to Generate Code

- ▶ Using AI Coding assistant (GitHub Copilot, Microsoft Copilot, ChatGPT)
- ▶ Prompt based Code Generation (Claude)
- ▶ Custom AI models (fine tuned GPT models)

<https://github.com/copilot/>

The screenshot displays the GitHub Copilot web interface. At the top, the browser address bar shows the URL `github.com/copilot/c/f7f4263c-d407-4840-bcb3-8a6f6b84a94a`. The Copilot header includes a search bar and a file explorer. The main workspace is titled "GPT-4o" and contains a prompt: "Generate classes for professor and student in Java using a super class". Below the prompt, a response from the AI states: "Sure! Here are the classes for `Professor` and `Student` in Java, utilizing a `Person` super class."

Three code snippets are shown in the workspace:

- Person.java**:

```
public abstract class Person {
    private String name;
    private String id;

    public Person(String name, String id) {
```
- Professor.java** (v2):

```
public class Professor extends Person {
    private String department;

    public Professor(String name, String id, String department) {
        super(name, id);
```
- Student.java**:

```
public class Student extends Person {
    private String department;

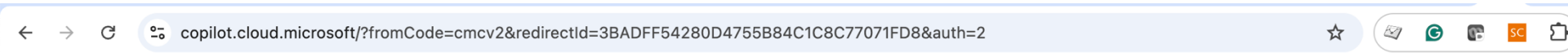
    public Student(String name, String id, String department) {
        super(name, id);
```

On the right, a code editor displays the full implementation of the `Person` class (22 lines, 380 bytes):

```
1 public abstract class Person {
2     private String name;
3     private String id;
4
5     public Person(String name, String id) {
6         this.name = name;
7         this.id = id;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public String getId() {
15        return id;
16    }
17
18    @Override
19    public String toString() {
20        return name + ", ID: " + id;
21    }
22 }
```

The bottom of the interface features an "Ask Copilot" button and a small icon.

<https://copilot.cloud.microsoft/>



Copilot

Generate classes for professor

New chat



Generate classes for professor and student in Java using a super class



Copilot

Sure! Here's an example of how you can create a superclass Person and then extend it to create Professor and Student classes in Java:

```
// Superclass
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}
```

Can you explain inheritance in Java?

How do I add methods to these classes?

What are interfaces in Java?

View prompts

Ask me anything...

0 / 8000



Copilot

Recent chats

Generate classes for p

[See more](#)

<https://gemini.google.com/>

← → ↻ 🔍 gemini.google.com/app/0e3db600a68c582a ☆ 📄 🌐 📱 📄

☰ Gemini ▾
2.0 Flash

Try Gemini Advan

+

Generate classes for professor and student in Java using a super class

Java

```
// Person.java (Superclass)
public class Person {
    private String id;
    private String name;
    private int age;
    private String address;

    public Person(String id, String name, int age, String address) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.address = address;
    }

    public String getId() {
        return id;
    }
}
```

Ask Gemini

+ 🔍 Deep Research 🔊

<https://chatgpt.com/>

chatgpt.com/c/67d6c650-cef4-8013-932b-fee776ff262

ChatGPT

ChatGPT

Generate classes for professor and student in Java using a super class

Here's a Java implementation where both `Professor` and `Student` inherit from a superclass

`Person` :

```
</> Professor Student

1 // Superclass
2 class Person {
3     protected String name;
4     protected int age;
5
6     public Person(String name, int age) {
7         this.name = name;
8         this.age = age;
9     }
10
11     public void displayInfo() {
12         System.out.println("Name: " + name + ", Age: " + age);
13     }
14 }
15
16 // Subclass: Professor
17 class Professor extends Person {
18     private String subject;
19
20     public Professor(String name, int age, String subject) {
21         super(name, age);
22         this.subject = subject;
23     }
24
25     @Override
26     public void displayInfo() {
```

Ask anything



<https://claude.ai/>

← → ↻ 📄 claude.ai/chat/6ad5e0ec-2958-48a3-8c39-b370114fdb0e



Claude 🗨️ Untitled ▾

GJ Generate classes for professor and student in Java using a super class

I'll create Java classes for Professor and Student using a common superclass. Here's a complete implementation:

java

📄 Copy

```
// Person superclass
public class Person {
    private String name;
    private int age;
    private String gender;
    private String address;

    // Constructor
    public Person(String name, int age, String gender, String address) {
        this.name = name;
        this.age = age;
        this.gender = gender;
        this.address = address;
    }

    // Getters and setters
    public String getName() {
        return name;
    }
}
```

GJ Reply to Claude...



📄 Claude 3.7 Sonnet ▾ 🎨 Choose style ▾

Concluzii

- ▶ Diagrame UML:
 - Interacțiuni
 - Comportamentale
 - Structură
- ▶ C4 Model: context, container, component, code

Întrebări

- ▶ 1) Până unde trebuie să modelați un proiect la care lucrați? (un proiect cu 4–5 persoane)
- ▶ 2) Cu ce ar trebui începută modelarea unui proiect?
- ▶ 3) E etic ca o firmă (mare) să facă RE pe un produs de la concurență?
- ▶ 4) De ce nu trebuie încurajat RE? Care sunt efectele RE?

Bibliografie

- ▶ Ovidiu Gheorghieș, Curs 5 IP
- ▶ www.uml.org
- ▶ Reverse Engineering and Design Discovery: A Taxonomy, Chikofsky, E.J. and Cross, J., January, 1990

Links (RE)

- ▶ **DJ Java Decompiler 3.10.10.93:**
<http://www.softpedia.com/progDownload/DJ-Java-Decompiler-Download-13481.html>
- ▶ **Open Office:** <http://ro.wikipedia.org/wiki/OpenOffice.org>
- ▶ **UML Reverse Engineering for Existing Java, C# , and Visual Basic .NET Code:**
<http://www.altova.com/umodel/uml-reverse-engineering.html>
- ▶ **Reverse Engineering:**
http://en.wikipedia.org/wiki/Reverse_engineering
- ▶ **PROTO 3000 3D Engineering Solutions:**
<http://www.proto3000.com/services.aspx>
- ▶ **HAR2009:** <http://www.degate.org/HAR2009/>
- ▶ **Degate:** <http://www.degate.org/screenshots/>
- ▶ **Intelligent:** <http://www.intelligenttrd.com/>
- ▶ **Smartphones RE:** <http://www.cytraxsolutions.com/2011/01/smartphones-security-and-reverse.html>